© 2021 LEI FAN

MACHINE LEARNING METHODS FOR PRICING AND HEDGING FINANCIAL DERIVATIVES

BY

LEI FAN

DISSERTATION

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Industrial Engineering
in the Graduate College of the
University of Illinois Urbana-Champaign, 2021

Urbana, Illinois

Doctoral Committee:

Assistant Professor Justin Sirignano, Chair
Professor Ramavarapu S. Sreenivas
Associate Professor Liming Feng
Assistant Professor Ruoyu Sun
Clinical Associate Professor Alexandra Chronopoulou

Abstract

Machine learning and deep learning have realized incredible success in areas such as computer vision and natural language processing. This thesis focuses on developing machine learning methods and models for financial data, which is a new application area in machine learning. It is challenging to successfully apply machine learning due to the substantial noise in financial data. As part of our approach to address this challenge of highly noisy data (which can cause machine learning models to easily overfit), we develop a pricing and hedging framework which merges classical stochastic differential equations (SDE) with neural networks. Optimization methods are developed to train these neural network-SDE models. The models are trained on large amounts of financial data using GPU high-performance computing.

The neural-network-based SDE models demonstrate superiority over traditional mathematical option pricing models such as the Black-Scholes model, the Dupire's local volatility model, and the Heston model. We develop a new optimization algorithm that can take unbiased, stochastic gradient descent steps for optimizing the neural-network-based SDE for mean-squared-error objective functions and European options. This algorithm allows for rapid optimization of the models. For training with general objective functions and more complex options (e.g., American options), a partial differential equation (PDE) is derived to evaluate the objective function and we then optimize over the PDE.

The training of models is mathematically and computationally challenging because one must optimize over an SDE/PDE for the European/American option price. Models are trained on large datasets (many contracts) and either large simulations (many Monte Carlo samples for the stock price paths) or large numbers of PDEs (a PDE must be solved for each contract). All numerical results are based on real market data including S&P500 index options, S&P100 index options, and single-stock American options. The neural-network-based SDE models are more accurate at out-of-sample pricing than the Black-Scholes model, the Dupire's local volatility model, and the Heston model. For out-of-the-money hedging, the neural-network-based SDE model outperforms Dupire's local volatility model and the Heston model while slightly underperforming the Black-Scholes model. For at-the-money hedging, the neural-network-based SDE model underperforms the other three models.

Finally, this thesis also explores modeling limit order book data with deep neural networks. In this area, the challenges become how to extract useful information from terabytes of limit order books data and carefully design deep neural networks to improve the prediction accuracy of stock prices. This thesis focuses on developing convolutional neural networks (CNN) and recurrent neural networks (RNN) to predict the mid-price direction for 0.1 and 1 second. The models are tested on over 100 stocks data and achieve superior performance than traditional machine learning models.

I'm dedicating this thesis to my family. A special feeling of gratitude to my loving parents, Yuemin Fan and Shubin Zhang who gave me continuous support and inspiration to pursue my dream. My brother, Zuotao Fan, also gave me endless encouragement throughout my career. I am truly thankful for having you in my life.

Acknowledgements

Throughout the writing of this dissertation, I have received a great deal of support and assistance.

First and foremost, I would like to express my sincere gratitude to my advisor, professor Justin Sirignano, for your patience, guidance, and support. I have benefited greatly from your assistance and dedicated involvement in every step throughout the process. I am extremely grateful that you took me on as a student and continued to have faith in me over the years.

I am also deeply grateful to the professors in ISE, professor Ramavarapu S. Sreenivas, professor Liming Feng, professor Ruoyu Sun, professor Alexandra Chronopoulou, and many others. Your immense knowledge and plentiful experience have encouraged me in all the time of my academic research and daily life. It's my great honor to be a member of ISE and learn from you during my Ph.D. study.

I would also like to thank my friends during my Ph.D. study. Your kind help and support made my study and life in the United States a wonderful time.

Contents

Chapter 1. Introduction **1**

1.1 Background and Motivation 1

1.2 Introduction to Options 2

1.3 Literature Review 3

1.4 Main Contributions and Challenges 6

Chapter 2. Neural-network SDE models for European Options **9**

2.1 The Black-Scholes Model 9

2.2 1-Dimensional Neural Network 11

2.3 Local Volatility Models 11

2.4 Stochastic Volatility Models 14

2.5 Algorithms 16

2.6 Experiments and Results 19

Chapter 3. Delta Hedging for European Options **35**

3.1 Introduction 35

3.2 The Next Day Hedging 37

3.3 At-The-Money Hedging 38

3.4 Hedging for Out-of-sample Payoff Functions 40

Chapter 4. Neural-network PDEs for American Options **43**

4.1 The Black-Scholes PDE model 46
4.2 1-Dimensional Neural-network PDE model 47
4.3 Neural-network Local Volatility PDE model 48
4.4 Stochastic Volatility PDE models 49
4.5 Experiments and Results 71

Chapter 5. Delta Hedging for American Options 94
5.1 The Next-day Hedging 94
5.2 The Next 5th Day Hedging 95

Chapter 6. Deep Learning Models for High-frequency Data 97
6.1 Limit Order Books Data and Selected Features 97
6.2 Models 98
6.3 Data Augmentation 100
6.4 Ensemble Models 101

Chapter 7. Conclusion 103

Bibliography 105

Chapter 1

Introduction

1.1 Background and Motivation

Machine learning (ML) was first introduced in 1959 by Arthur Samuel [35]. It is a branch of artificial intelligence (AI) that develops computer systems that can improve themselves adaptively from data and automatically identify patterns or make decisions without the help of human beings. Traditional machine learning models include decision trees, support vector machines (SVM), k-nearest neighbors, and regression analysis. Deep learning (DL) [27] is a recent class of machine learning algorithms that have achieved substantial success in recent years. Deep learning uses artificial neural networks (ANNs) with multiple layers to extract features from data.

The performance of machine learning models not only depends on the development of novel architectures (such as deep neural networks) but also heavily relies on the quality of the data. A good dataset should contain sufficient information, with low noise and unambiguous labels. For example, ImageNet [11] and CIFAR-10 [26] are well-known datasets in image recognition and are generally used as benchmarks to compare the performance of various models. There is also an increasing number of

other high-quality datasets, which have facilitated the development of machine learning and deep learning models.

However, in the area of finance, the story is not as simple. Financial data generally contains missing data, a large amount of noise, and limited information in the market. In addition, the financial data stream is a function of time, and the use of future information is strictly forbidden for prediction. Due to these disadvantages, it is challenging to develop successful machine learning models in finance.

This research develops machine learning methods and models for two important applications in financial science and industry: pricing and hedging financial derivatives and predicting asset prices.

1.2 Introduction to Options

Options are contracts between two parties that give the buyer the right to buy or sell the underlying asset at a specific price on or before a certain date. The specific price is called the strike price and the time to execution is maturity. The buyer can choose whether to execute the contract or not to maximize his profit. Options have value and their prices are closely related to their corresponding asset prices.

There are two types of options: call options and put options. Call options give the buyer the right to buy the underlying asset at the strike price upon execution. Typically when the underlying asset price is over the strike price, the buyer will choose to execute the contract to earn the premium. However, if the underlying asset price is below the strike price, the buyer will not execute the option and its price is 0. The put option has similar functionality except that it allows the buyer to sell the underlying asset at the strike price upon execution.

The payoff functions for the call and put option when being executed are

$$f_c = \max(S_T - K, 0)$$

$$f_p = \max(K - S_T, 0)$$

where f_c, f_p are the payoff for call and put options. S_T is the underlying asset price and K is the strike price.

There are multiple styles of options. Two of the most important styles are the European style and the American style. The former means the option can only be executed on the maturity date, while the latter means the option can be executed any time on or before the maturity date.

So the option price for the European option is

$$P_t^{European} = \mathbb{E}\left[e^{-r(T-t)} f(S_T)\right] \tag{1.1}$$

where $P_t^{European}$ is the option price and $f()$ is the payoff function.

The option price for the American option is

$$P_t^{American} = \sup_{\textit{All possible strategies for selecting } \tau \in [t,T]} \mathbb{E}\left[e^{-r(\tau-t)} f(S_\tau)\right] \tag{1.2}$$

where $P_t^{American}$ is the option price and $f()$ is the payoff function.

1.3 Literature Review

Options pricing is a crucial research area in quantitative finance. Numerous papers have explored theoretical pricing models with different assumptions and calibrated the models to market prices. Traditional numerical pricing approaches include the

tree models [8], Monte Carlo simulation [4], and finite difference methods [5, 36]. In 1973, Black, Scholes, and Merton [3,32] developed the analytical solution for European options with the assumptions of risk-neutral pricing and that the underlying asset price follows a Geometric Brownian process. The Black-Scholes formula has been historically widely used in options trading [29].

However, not all of the assumptions in the Black-Scholes model are empirically valid. For example, researchers found that the smiles (skewness) and term structures in observed implied volatilities could not be explained with the lognormal assumption of asset price returns. Clearly, these deficiencies should be resolved with more complex models. Jumps were introduced by Merton [33] to describe the non-continuous movements of asset prices. Such jump-diffusion models have been further explored in the literature [25]. On the other hand, other papers have focused on modeling volatility. Some papers regard the volatility itself as a stochastic process, which leads to the class of stochastic volatility models [19, 20, 38]. The volatility can also depend on time and the state of the asset price process, and the concept of local volatility was developed by Dupire, Derman, and Kani [10, 12].

Neural networks were introduced to option pricing and hedging in the early 1990s [22, 30, 31]. Due to the universal approximation theorems [9, 21], neural networks can approximate any continuous functions and thus capture nonlinear dependency between inputs and outputs. Kelly [23] investigated the pricing of American put options and demonstrated that ANNs performed much better than the competing Cox-Ross-Rubinstein market model (CRR model). Carverhill and Cheuk [6] also achieved similar conclusions on S&P 500 index futures options (calls and puts). Healy et al. [18] confirmed that the performance of ANN is consistent with the Black-Scholes formula on FTSE 100 European call options. Most of these papers focus on proving the effectiveness of ANNs in various datasets. These papers directly train a neural network on a

target function or dataset (i.e., regression), which is different from our approach.

More recently, Funahashi [14] presented a new approach to option pricing combining artificial neural networks and an asymptotic expansion. Specifically, approximated option price could be obtained by calculating a truncated sum of iterated Ito stochastic integrals. Then an artificial neural network was used to learn the residual between the approximated value and the real value. Some other papers tried to directly model option prices by carefully selecting features without using historical or implied volatility. Thus, complicated stochastic differential equations (SDEs) were avoided. In [16], the author proposed three different data-driven approaches (feature sets) for pricing European-style call options using XGBoost and artificial neural networks. The features consist of log returns, time to maturity, interest rate, previous option prices, and covariance matrix of open, high, low, and close price of the underlying asset. Although neither historical nor implied volatility was used, the results implied that the trained models performed better or were similar to the Black-Scholes formula.

In the recent paper [15] which appeared a few months ago, the authors proposed an approach that combines neural networks with classical SDEs. The neural networks were used to replace coefficient functions within the SDE, to improve the accuracy of the overall pricing model. They refer to their approach as a "Neural SDE". The authors present the algorithms to calibrate the Neural SDE, as well as propose an algorithm to find robust price bounds for an illiquid derivative. In spirit, their approach has some similarities to ours. However, we use different models and optimization methods. In our paper, we propose a universal option pricing approach with neural networks that can be optimized on all available contracts (call and put options with different maturity and strike prices). A major difference in our approach is we optimize over the partial differential equation (PDE) governing the price of the financial derivative, which allows us to use a much more general class of objective functions and calibrate

our model to a much wider class of financial derivatives (such as American options). In addition, they use simulated data which is generated by the Heston model, while we use real market data to calibrate and evaluate our models.

This thesis also explores modeling limit order book data with deep learning models. Many papers have studied the characteristics of limit order books [1, 2, 17]. There are also recent papers that develop deep learning models for limit order books. For example, Sirignano [37] developed a spatial neural network that allows for effective use of information from deep in the order book. Long et al. [28] proposed a multi-filters neural network for feature extraction and price move prediction tasks. In this thesis, we develop convolutional neural networks (CNN) and recurrent neural networks (RNN) to predict price changes using limit order book data.

1.4 Main Contributions and Challenges

The main contributions of this paper include:

- Propose neural-network-based SDE models that demonstrate improved pricing accuracy compared to traditional mathematical option pricing models. See figure 1.1 below.
- Develop a new stochastic gradient descent (SGD) algorithm for calibrating neural-network SDE models given European option price data.
- Develop a calibration approach using the PDE representation for the neural-network SDE model.
- The neural-network-based PDE methods allow us to calibrate the neural-network SDE model to American option market price data.

- Develop and evaluate deep learning models for high-frequency limit order book data.

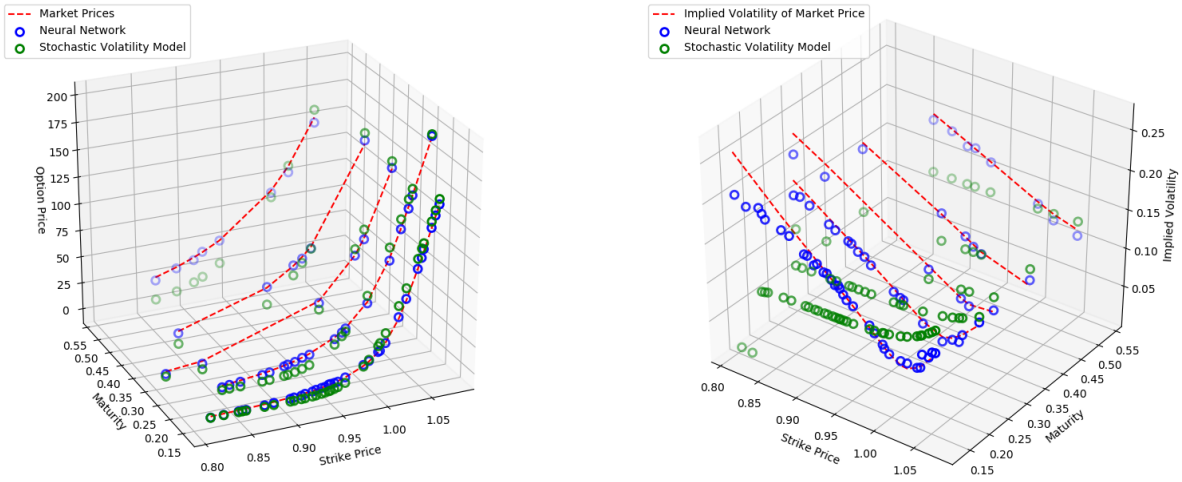


Figure 1.1: Out-of-sample comparison of the stochastic volatility model with neural-network SDE model for put options on 2017-09-01. The left figure compares the model predictions for the option prices, while the right figure compares the corresponding implied volatilities.

There are various challenges that we have addressed in the research. Two main challenges for options pricing and hedging are:

- Models are calibrated by optimizing over an SDE or PDE for the European/American option price. This is a computationally challenging optimization problem.

- Optimizing over large datasets (many contracts) and large simulations (many Monte Carlo samples for the stock price paths).

In the research on limit order books, the challenges include how to extract useful information from terabytes of data and carefully design deep neural networks to improve the prediction accuracy for stock prices.

This dissertation is organized as follows. In Chapter 2, we first briefly review the general approach for option pricing and some of the most common pricing models, such as the Black-Scholes model, the Dupire's local volatility model, and the Heston stochastic volatility model. Then, we propose the neural-network-based SDE models. We implement and evaluate the models on European option datasets, demonstrating that the neural-network-based models have superior pricing accuracy in comparison to these traditional mathematical options pricing models. In addition, in chapter 3, we discuss delta hedging for these models. In order to calibrate the neural-network SDE model, one must optimize over the SDE, which is computationally challenging. We derive a new unbiased stochastic gradient descent algorithm that can efficiently perform this optimization. In Chapter 4, we derive a PDE model for option prices from the neural-network SDE. The PDE model, which is a Kolmogorov backward equation (KBE), can be used to calibrate the neural-network SDE. We optimize over a finite-difference equation approximating the PDE using automatic differentiation. The PDE model allows for calibration using a much wider class of objective functions than the stochastic gradient descent approach. Furthermore, the PDE model allows for calibration on American options, while the stochastic gradient descent approach does not. We calibrate the neural-network PDE models to the S&P 100 options data and 100 American stock options data. The hedging performance for the neural-network PDE models is discussed in chapter 5. Finally, in Chapter 6, we summarize our research on deep learning models for limit order book data.

Chapter 2

Neural-network SDE models for European Options

In this chapter, we will first discuss some of the most common options pricing models which include the Black-Scholes model, the local volatility model, and the stochastic volatility model. Then the neural-network-based SDE models are proposed. By comparing the performance on European option datasets, we demonstrate that the neural-network-based models have superior pricing accuracy in comparison to these traditional mathematical options pricing models.

2.1 The Black-Scholes Model

Consider the price S_t of an asset at time t which satisfies the stochastic differential equation:

$$dS_t = \mu S_t dt + \sigma S_t dW_t \tag{2.1}$$

where W_t is a standard Wiener process, μ is the percentage drift, and σ is the percentage diffusion or volatility. μ and σ are constant. μ is often replaced by $r - d$, where r is

the risk-free interest rate and d is the dividend rate. (2.1) is referred to as a geometric Brownian Motion (GBM).

Under the assumptions of risk-neutral and constant μ and σ, Black and Scholes [3] developed the pricing formula for European options, which is known as the Black-Scholes formula.

$$C_t = S_t e^{-d\tau} N(d_1) - K e^{-r\tau} N(d_2) \tag{2.2}$$

$$P_t = K e^{-r\tau} N(-d_2) - S_t e^{-d\tau} N(-d_1) \tag{2.3}$$

where

$$d_1 = \frac{ln(S_t/K) + (r - d + \sigma^2/2)\tau}{\sigma\sqrt{\tau}} \tag{2.4}$$

$$d_2 = d_1 - \sigma\sqrt{\tau} \tag{2.5}$$

C_t and P_t are call and put option prices. S_t is the underlying asset price and K is the strike price. τ is time to maturity and equals $T - t$. $N(x)$ is the cdf of the standard normal distribution.

However, in real market data, r and σ are generally not constant, which makes the Black-Scholes model inaccurate in pricing options.

The SDE (2.1) can be discretized using the Euler scheme:

$$S_{t+1} = S_t + \mu S_t \Delta t + \sigma S_t \sqrt{\Delta t} Z_t \tag{2.6}$$

where Z_t are i.i.d. standard normal random variables. (2.6) can be simulated using Monte Carlo simulation and then used to price financial derivatives.

Financial derivatives are priced under the risk-neutral measure, which is equivalent

to setting the drift μ in (2.6) to risk-free rate r. The call and put option prices at time t for European options are:

$$C_t = \mathbb{E}\left[e^{-r(T-t)} \max(S_T - K, 0)\right] \tag{2.7}$$

$$P_t = \mathbb{E}\left[e^{-r(T-t)} \max(K - S_T, 0)\right] \tag{2.8}$$

We will introduce a series of models in the following subsequent sections which model μ and σ as functions of S_t and t.

2.2 1-Dimensional Neural Network

Consider the percentage diffusion σ in equation (2.1) as a function of the current underlying asset price S_t.

$$dS_t = (r - d)S_t dt + \sigma(S_t)S_t dW_t \tag{2.9}$$

We can use a 1-dimensional neural network (1DNN) to learn the function $\sigma(S_t)$ (i.e., the input and output are scalars).

2.3 Local Volatility Models

In local volatility models, the volatility σ is treated as a function of both the current asset price S_t and time t. The SDE for the local volatility model is

$$dS_t = (r - d)S_t dt + \sigma(S_t, t)S_t dW_t \tag{2.10}$$

2.3.1 Dupire Model

The local volatility function can be deduced from the quoted call and put options in the market using Dupire's formula [13]:

$$\sigma^2(K,T) = 2\frac{\frac{\partial C}{\partial T} + (r-d)K\frac{\partial C}{\partial K} + dC}{K^2\frac{\partial^2 C}{\partial K^2}}, \tag{2.11}$$

where K is the strike price and T is the time to maturity in a contract.

In practice, the implied volatility surface observed in the market must be interpolated since only a discrete number of contracts are available, and then the interpolated function is substituted into Dupire's formula. Using equation (2.11), one can simulate the SDE (2.10).

Below are the steps to implement the local volatility model [10].

- Assemble the data, consisting of a matrix of quoted option prices $\{C(T_i, K_j^i)\}_{i=1}^N$ where $j = 1, 2, \ldots, M_i$.
- Use cubic spline interpolation to produce a smooth volatility surface.
- Calculate $\sigma(K,T)$, the local volatility surface, using Dupire's formula where

$$\frac{\partial C}{\partial T} \approx \frac{C(K,T+\Delta T) - C(K,T-\Delta T)}{2\Delta T} \tag{2.12}$$

$$\frac{\partial C}{\partial K} \approx \frac{C(K+\Delta K,T) - C(K-\Delta K,T)}{2\Delta K} \tag{2.13}$$

$$\frac{\partial^2 C}{\partial K^2} \approx \frac{C(K+\Delta K,T) - 2C(K,T) + C(K-\Delta K,T)}{(\Delta K)^2} \tag{2.14}$$

- Use equation (2.6) and Monte Carlo simulation to calculate the option prices.

So based on the above observation, the function $\sigma(S_t, t)$, or the local volatility surface, is produced from the implied volatility surface and the Dupire's formula.

2.3.2 Neural-network Local Volatility Model

Instead of the indirect method that is using implied volatility surface to generate local volatility surface, we can use a neural network to learn the call option surface directly. We call this model the neural-network local volatility (NNLV) model. There are three steps.

- **Step 1:** Define a neural network $f(s, t)$ whose inputs are the stock price s and time t. Train the neural network to minimize the objective function $\sum_{K_i,T_i} (P_{K_i,T_i} - f(K_i, T_i))^2$ where (K_i, T_i, P_{K_i,T_i}) are the strike prices, maturities, and option prices observed in the market.

- **Step 2:** Using Dupire's formula and the finite difference method on the neural network, we can get the local volatility function for the SDE.

- **Step 3:** Use equation (2.6) and Monte Carlo simulation to calculate the option prices.

2.3.3 SDE-optimized Neural Network

Instead of optimizing the neural network at step 1, we can optimize it over all the steps. That is, we optimize over the entire distribution of the SDE random variable. This is a much more sophisticated optimization approach (and also substantially more computationally expensive). In order to optimize the SDE distribution, we modify step 1 and 3 from the previous method:

- **Step 1:** Define a neural network $f(s,t)$ whose inputs are the stock price s and time t. The output of the neural network is the call option surface in the SDE.

- **Step 3:** Use equation (2.6) and Monte Carlo simulation to calculate the option prices. Train the neural network to minimize the mean squared error between the model's outputs for the prices (from the Monte Carlo simulation of the SDE) and the true option prices in the dataset.

We call this model the SDE-optimized Neural Network (SDENN) model.

Besides optimizing the neural network over the SDE, the drift can be modeled as another independent neural network $\mu(S_t,t)$, which can be trained together with $f(S_t,t)$. This model is called 'SDENN-Drift'.

Instead of using Dupire's formula, we can also directly use a neural network for the local volatility. Define a neural network $\sigma(S_t,t)$ that outputs the local volatility. Use equation (2.6) and Monte Carlo simulation to calculate the option prices. Train the neural network that minimizes the mean squared error of simulated and true option prices. We call this model 'SDENN-LV', where '-LV' refers to the local volatility.

2.4 Stochastic Volatility Models

The stochastic volatility models regard diffusion as another stochastic process. The most well-known stochastic volatility model is the Heston model.

2.4.1 Heston Model

The mathematical equations for the Heston model are below:

$$dS_t = \mu S_t dt + \sqrt{Y_t} S_t dW_t^S \tag{2.15}$$

$$dY_t = \alpha(m - Y_t)dt + k\sqrt{Y_t}dW_t^Y \tag{2.16}$$

where W_t^S and W_t^Y are Wiener processes with correlation ρ. μ is the drift. Y_0, α, m, k, ρ are parameters for the Heston model. More specifically:

- Y_0 is the initial value for Y_t.
- m is the long variance. As t tends to infinity, the expected value of Y_t tends to m.
- α is the rate at which Y_t reverts to m.
- k is the volatility of the volatility and determines the variance of Y_t.

We correspondingly estimate Y_0, α, m, k, ρ based on the optimization algorithm. Moreover, if the parameters satisfy the Feller condition $2\alpha m > k^2$, the process Y_t is strictly positive.

2.4.2 2-Dimensional Neural Network SDE with Correlated Wiener Processes

Let $f(S_t, Y_t, t)$ represents for a neural network that the input and output dimensions are 3 and 4, respectively. Y_t is another random variable with its own SDE. Let f_i represents for the i-th (i=1,2,3,4) output of the neural network. The mathematical equations for the 2-dimensional neural network (2D-NN) model are:

$$dS_t = f_1(S_t, Y_t, t)dt + f_2(S_t, Y_t, t)dW_t^S \tag{2.17}$$

$$dY_t = f_3(S_t, Y_t, t)dt + f_4(S_t, Y_t, t)dW_t^Y \tag{2.18}$$

W_t^S and W_t^Y are two standard Wiener processes with correlation ρ, where ρ are trained together with the neural networks. Besides, we add the initial value Y_0 ($Y_0 \geq 0$) as the additional variable to train.

2.5 Algorithms

We develop an unbiased stochastic gradient descent algorithm for optimizing the SDE model. Below is the sample algorithm for the model in section 2.3.3. The algorithms for the other models are similar.

Consider f_1 is a neural network, and f_2 is a deep copy of f_1. The algorithm is below.

Algorithm 1 Optimization algorithm

procedure
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $f_2 \leftarrow deepcopy(f_1)$
    Set f_2 parameters $requires_grad = False$.
    $S_1 \leftarrow ones(N, L)$
    for $t \leftarrow 1$ `to` M **do**
      $C_1^{NN} \leftarrow f_1(S_1, t)$
      $\sigma_1 \leftarrow g(C_1, S_1, t)$
      $S_1 \leftarrow S_1 + (r - d)S_1\Delta + \sigma_1 S_1\sqrt{\Delta}Z_t$
    end for
    $S_2 \leftarrow ones(N, L)$
    for $t \leftarrow 1$ `to` M **do**
      $C_2^{NN} \leftarrow f_2(S_2, t)$
      $\sigma_2 \leftarrow g(C_2, S_2, t)$
      $S_2 \leftarrow S_2 + (r - d)S_2\Delta + \sigma_2 S_2\sqrt{\Delta}Z_t$
    end for
    $C_1 \leftarrow \frac{1}{L}\sum_{i=1}^{L}[e^{-rT}max(S_1^i - K/S_0, 0)]$
    $C_2 \leftarrow \frac{1}{L}\sum_{i=1}^{L}[e^{-rT}max(S_2^i - K/S_0, 0)]$
    $P_1 \leftarrow \frac{1}{L}\sum_{i=1}^{L}[e^{-rT}max(K/S_0 - S_1^i, 0)]$
    $P_2 \leftarrow \frac{1}{L}\sum_{i=1}^{L}[e^{-rT}max(K/S_0 - S_2^i, 0)]$
    $loss \leftarrow \sum((C/S_0 - C_1)(C/S_0 - C_2) + (P/S_0 - P_1)(P/S_0 - P_2))$
    loss.backward()
    optimizer.step()
  end for
end procedure

where C_1^{NN} and C_2^{NN} are call option prices from the neural networks. $g(C, S, t)$ is Dupire's formula. C and P are market prices for call and put options.

The algorithm uses two independent simulation paths of the SDE to calculate an unbiased estimate of the gradient of the objective function. This method is used to give an unbiased estimator of the gradient.

Consider the objective function

$$J(\theta) = \sum_{T,K} \left(P_{model}(K, T; \theta) - P_{market}(T, K)\right)^2 \tag{2.19}$$

where $P_{model}(K,T;\theta) = \mathbb{E}[(S_T - K)^+]$ where S_t satisfies a neural-network SDE model with parameters θ. $P_{market}(T,K)$ is the observed market price data for the contract with maturity T and strike price K.

For simplicity, let's consider a single maturity and strike price. Then

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta (E[(S_T - K)^+] - P_{market}(T,K))^2 \\ &= 2\left(E[(S_T - K)^+] - P_{market}(T,K)\right) \nabla_\theta E[(S_T - K)^+]\end{aligned} \tag{2.20}$$

A naive Monte Carlo estimate of the gradient $\nabla_\theta J(\theta)$ above is given by:

$$S^N = 2(\frac{1}{N}\sum_{i=1}^{N}(S_T^i - K)^+ - P_{market}(T,K)) \times \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \left[(S_T^i - K)^+\right], \tag{2.21}$$

where S_t^i are i.i.d. Monte Carlo paths for the SDE S_t. By the Law of Large Numbers, $S^N \to \nabla_\theta J(\theta)$ as $N \to \infty$. However, for finite N, the estimate is biased:

$$\mathbb{E}[S^N] \neq \nabla_\theta J(\theta) \tag{2.22}$$

This is due to the correlation between $(\frac{1}{N}\sum_{i=1}^{N}(S_T^i - K)^+ - P_{market}(T,K))$ and $\frac{1}{N}\sum_{i=1}^{N} \nabla_\theta [(S_T^i - K)^+]$.

Consequently, S^N is a biased estimator. In order to increase its accuracy, a large number of samples N is required.

An unbiased estimate for $\nabla_\theta J(\theta)$ is

$$V^N = 2(\frac{1}{N}\sum_{i=1}^{N}(S_T^i - K)^+ - P_{market}(T,K)) \times \frac{1}{N}\sum_{j=N+1}^{2N} \nabla_\theta \left[(S_T^j - K)^+\right] \tag{2.23}$$

where $\mathbb{E}[V^N] = \nabla_\theta J(\theta)$ for any $N \in \mathbb{N}$.

V^N is an unbiased estimator of the gradient $\nabla_\theta J(\theta)$. Consequently, it allows for

computationally efficient optimization using stochastic gradient descent. For example, the SDE model can be optimized with stochastic gradient descent using $N = 2$ Monte Carlo paths of the SDE S_t at each optimization iteration. In contrast, the estimator S^N requires a large number of Monte Carlo samples at each optimization iteration in order to be accurate.

The corresponding simulation algorithm is below.

Algorithm 2 Simulation algorithm

procedure
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $S \leftarrow ones(N, L)$
    for $t \leftarrow 1$ `to` M **do**
      $\sigma \leftarrow f_1(S, t)$
      $S \leftarrow S + (r - d)S\Delta + \sigma S\sqrt{\Delta}Z_t$
    end for
    $C \leftarrow \frac{1}{L}\sum_{i=1}^{L}[e^{-rT}max(S^i - K/S_0, 0)]$
    $P \leftarrow \frac{1}{L}\sum_{i=1}^{L}[e^{-rT}max(K/S_0 - S^i, 0)]$
  end for
end procedure

2.6 Experiments and Results

2.6.1 Neural Network Structure and Dataset

We use a multilayer perceptron (MLP) with two hidden layers. Each hidden layer consists of 200 hidden units. The activation function is the softplus function with $\beta = 1$.

The dataset[1] consists of contracts from 5 trading days. We select the date 2017-09-01 and a random date from each of the subsequent months (2017-10-23, 2017-11-10, 2017-12-07, and 2018-01-26). In addition, we select the contracts that have a maturity

[1]Source: Option Metrics in Wharton Research Data Service (WRDS).

between a month and a year. There are about 800 pairs of call and put option contracts for each date. Each pair of call and put option contracts has the same strike price and maturity.

We split the data into training and test sets. The training set contains 80% of the contracts while the test set contains the remaining 20% of the contracts.

2.6.2 Evaluation Metrics

Some evaluation metrics are:

- **Mean squared error:** The mean squared error of predicted and true option prices.

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i^{true} - y_i^{pred})^2$$

- **Mean absolute error rate:**

$$r = \frac{1}{N}\sum_{i=1}^{N}\frac{|y_i^{true} - y_i^{pred}|}{y_i^{true}} \times 100\%$$

2.6.3 Intraday Out-of-Sample Prediction

Below are the tables that contain the results of the testing set. We can observe that neural-network-based models perform better than traditional option pricing models, and that the 2-dimensional neural network achieves the best result (See Figure 2.1).

Summary of results for intraday out-of-sample prediction, Part 1.								
Days	BS		Dupire		NNLV		SDENN	
	call	put	call	put	call	put	call	put
2017-09-01	118.41 102.79%	53.87 64.76%	121.92 97.61%	15.24 60.69%	93.61 26.38%	14.98 49.14%	96.63 19.13%	15.58 48.50%
2017-10-23	128.34 56.54%	92.56 69.38%	148.08 40.69%	30.53 64.18%	114.79 9.39%	37.27 60.09%	115.01 11.97%	30.31 59.34%
2017-11-10	123.10 79.39%	90.47 67.71%	111.42 68.33%	37.28 63.74%	91.33 14.16%	46.27 60.16%	94.45 14.71%	39.16 57.75%
2017-12-07	79.90 58.31%	47.69 68.31%	69.30 46.93%	24.33 67.73 %	60.57 12.37%	49.48 70.75%	64.04 12.98%	33.02 68.17%
2018-01-26	213.96 30.12%	133.70 65.31%	200.93 20.69%	46.18 62.43%	189.98 14.16%	52.85 60.16%	190.41 18.18%	49.04 59.75 %

Table 2.1: Summary of results for intraday out-of-sample prediction, Part 1. The first row is the mean squared error and the second row is the mean absolute error rate. "BS" means the Black-Scholes model. "Dupire" means Dupire's model. "NNLV" is the neural-network local volatility model, and "SDENN" refers to the SDE-optimized neural network.

Summary of results for intraday out-of-sample prediction, Part 2.						
Days	SDENN-Drift		2D-NN		Heston	
	call	put	call	put	call	put
2017-09-01	26.91 32.27%	6.23 44.95%	9.51 8.83%	2.90 41.10%	73.47 55.45%	49.18 66.29%
2017-10-23	11.87 11.03%	13.42 54.49%	7.73 5.91%	3.07 42.84%	182.51 30.30%	79.02 72.27%
2017-11-10	17.03 16.31%	13.07 49.63%	6.59 7.00%	2.20 40.97%	70.16 53.03%	74.69 67.88%
2017-12-07	33.88 13.73%	16.73 65.64%	5.89 9.42%	1.80 41.95 %	196.20 40.77%	48.94 71.21%
2018-01-26	21.88 13.26%	22.39 54.91%	10.70 8.20%	10.10 36.34%	132.97 15.71%	97.78 65.67%

Table 2.2: Summary of results for intraday out-of-sample prediction, Part 2. The first row is the mean squared error and the second row is the mean absolute error rate. "SDENN-Drift" is the SDE-optimized neural network with the drift additionally modeled as another neural network, while "2D-NN" refers to the 2-dimensional neural networks. "Heston" refers to the Heston model.

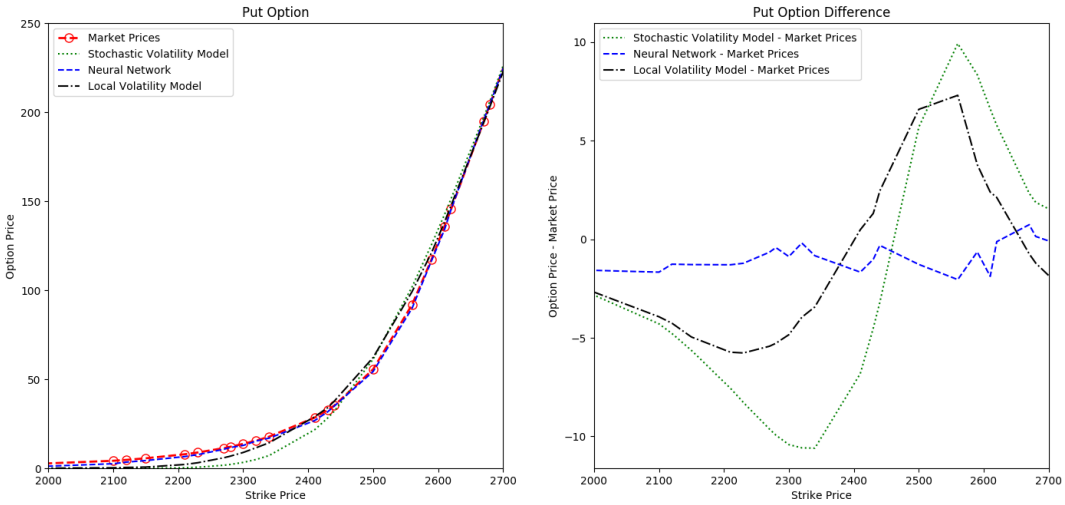


Figure 2.1: Comparison of neural-network SDE model, stochastic volatility model, and local volatility model for put options on 2017-09-01. The time to maturity is 77 days. The left figure displays the intraday out-of-sample prediction for the put option prices. The right figure shows the difference between the predicted prices and the actual market prices.

2.6.4 The Next-day Prediction

In this section, we use the models that are trained on 80% of the data and predict the option prices for the next trading day.

Summary of results for the next-day prediction, Part 1 (out of sample).								
Days	BS		Dupire		NNLV		SDENN	
	call	put	call	put	call	put	call	put
2017-09-05	488.56 110.87%	172.73 69.29%	330.08 105.79%	73.78 64.46%	305.63 34.69%	106.37 59.33%	321.66 40.27%	92.31 57.00%
2017-10-24	101.33 52.33%	91.90 72.01%	157.92 58.40%	32.37 67.29%	121.08 17.96%	37.88 65.07%	128.29 26.78%	29.79 62.26%
2017-11-13	88.15 65.15%	84.21 68.90%	54.87 55.05%	39.59 66.42%	42.38 16.75%	26.01 59.24%	40.80 16.94%	20.69 56.35%
2017-12-08	125.17 49.46%	112.01 69.79%	53.90 53.39%	39.41 66.56%	48.33 11.39%	38.69 67.34%	42.30 11.19%	23.21 64.12%
2018-01-29	527.35 29.53%	209.04 70.34%	349.77 18.68%	117.23 68.62 %	332.92 9.78%	142.82 66.61%	338.95 11.99%	133.33 66.04%

Table 2.3: Summary of results for the next-day prediction, Part 1 (out of sample). The first row is the mean squared error and the second row is the mean absolute error rate. The notation for different models is the same as table 2.1.

Summary of results for the next-day prediction, Part 2 (out of sample).						
Days	SDENN-Drift		2D-NN		Heston	
	call	put	call	put	call	put
2017-09-05	169.12 37.25%	34.13 48.02%	121.93 11.20%	52.46 51.70%	291.25 64.84%	142.13 70.24%
2017-10-24	13.20 20.18%	11.74 58.16%	8.30 6.65%	2.78 48.06%	136.79 29.86%	72.27 73.35%
2017-11-13	39.62 20.89%	21.31 49.27%	40.11 11.69%	9.89 43.22%	68.80 42.68%	74.26 69.48%
2017-12-08	71.02 12.20%	20.21 61.81%	11.78 8.07%	9.21 39.39%	67.09 35.33%	52.40 68.96%
2018-01-29	75.55 9.09%	59.54 61.94%	55.72 9.74%	24.51 45.92%	347.56 17.33%	173.25 71.50%

Table 2.4: Summary of results for the next-day prediction, Part 2 (out of sample). The first row is the mean squared error and the second row is the mean absolute error rate. The notation for different models is the same as table 2.2.

2.6.5 Recalibration

In addition to the next-day prediction, we'd like to explore the performance of our models over a longer time period. Here the models are calibrated using a predetermined

method.

Specifically, we select the dataset over 2 months period (September and October of 2017) which includes 42 trading days. On each day, there are about 1100 contract pairs (call and put options). Moreover, for each day, we randomly split the contracts into training and testing sets with a ratio of 0.8:0.2.

We use the 2-dimensional neural network model to do our experiments. We train the model on 2017-09-01 (the first day of the dataset).

Now we consider the following two experiments:

- **Pre-train:** We train the model on 2017-09-01 (the first day of the dataset) and test its performance on all the testing data for the 42 trading days.

- **Recalibration:** On day t, we use the pretrained model from day t-1 (the previous trading day) and train the model on the training data on day t. We use Adam with a learning rate of 1e-5 and 100 epochs. We test the model performance on the corresponding testing set.

Firstly, we compare the averaged performance of these two methods on the testing data.

Comparison of Recalibration on Testing Data.				
Days	Pre-train		Recalibration	
	Call	Put	Call	Put
2017-09-01∼ 2017-10-31	3.684 34.328 9.439%	2.765 19.197 48.916%	3.230 32.162 8.292%	2.109 10.243 47.874%

Table 2.5: Comparison of the averaged performance of pre-train and recalibration methods for two months. There are three columns for each cell. The first column is for "MAE". "MAE" means mean absolute error. The second column is for "MSE". "MSE" means mean squared error. The third column is for "relative MAE", which means the relative mean absolute error.

We can observe that the recalibration method outperforms the pre-trained method

in every evaluation metric, which proves that this method is valid for the long time period.

Moreover, we explore the performance of these two methods on specific days. In the following table, we compare the recalibration for 1 day, 10 days, 20 days, 40 days after the first day (2017-09-01). The corresponding dates are 2017-09-05, 2017-09-18, 2017-10-02, and 2017-10-30.

Comparison of Recalibration for Specific Days.				
Days	Pre-train		Recalibration	
	Call	Put	Call	Put
2017-09-05 (1 day)	3.775 48.218 9.228%	3.748 29.487 52.143%	3.795 55.767 8.094%	2.465 15.312 47.585%
2017-09-18 (10 day)	3.727 37.938 11.331%	2.611 16.222 42.370%	3.406 41.895 8.124%	2.284 11.604 42.097%
2017-10-02 (20 day)	5.726 53.764 8.056%	2.742 16.608 45.494%	5.009 49.031 6.875%	2.269 10.244 46.690%
2017-10-30 (40 day)	3.320 27.170 9.509%	2.843 22.104 51.609%	2.810 24.076 8.947%	1.850 6.866 49.179%

Table 2.6: Comparison of recalibration for specific days. There are three columns for each cell. The first column is for "MAE". "MAE" means mean absolute error. The second column is for "MSE". "MSE" means mean squared error. The third column is for "relative MAE", which means the relative mean absolute error.

We can still observe that the recalibration method outperforms the pre-trained method in most evaluation metrics. Below are the corresponding figures for these four days. We can conclude that both methods calibrate the market prices very well.

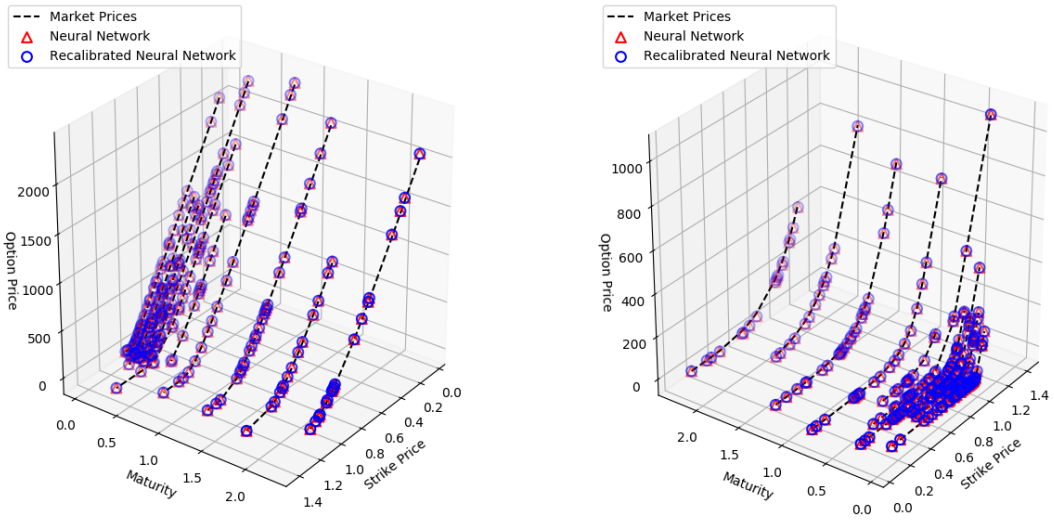


Figure 2.2: We compare the model performance for fitting the out-of-sample option curve on the 1st day (2017-09-05). The left figure is for call options and the right figure is for put options. "Market Prices" represents the true option prices. "Neural Network" represents the model without recalibration, while the "Recalibrated Neural Network" represents the model after recalibration.

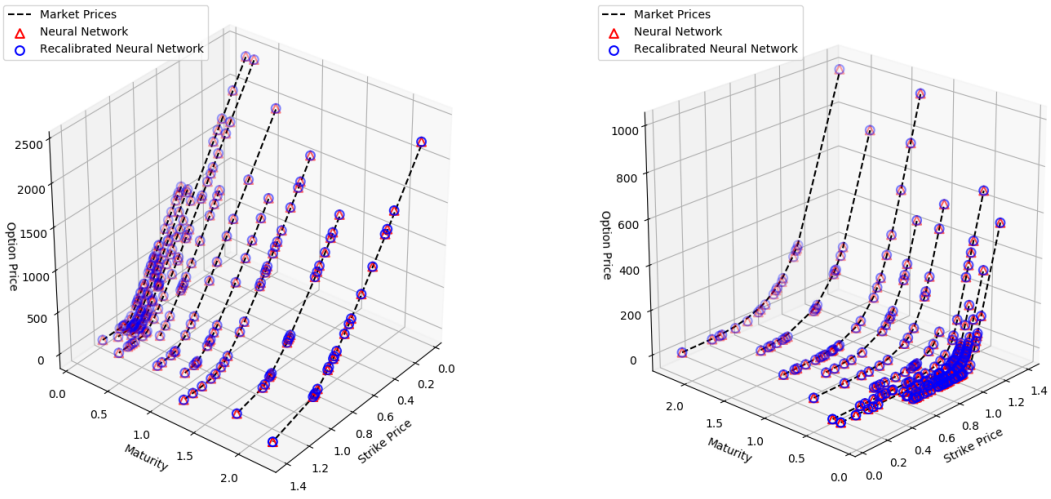


Figure 2.3: We compare the model performance for fitting the out-of-sample option curve on the 10th day (2017-09-18). The notation for different models is the same as the previous figure 2.2.

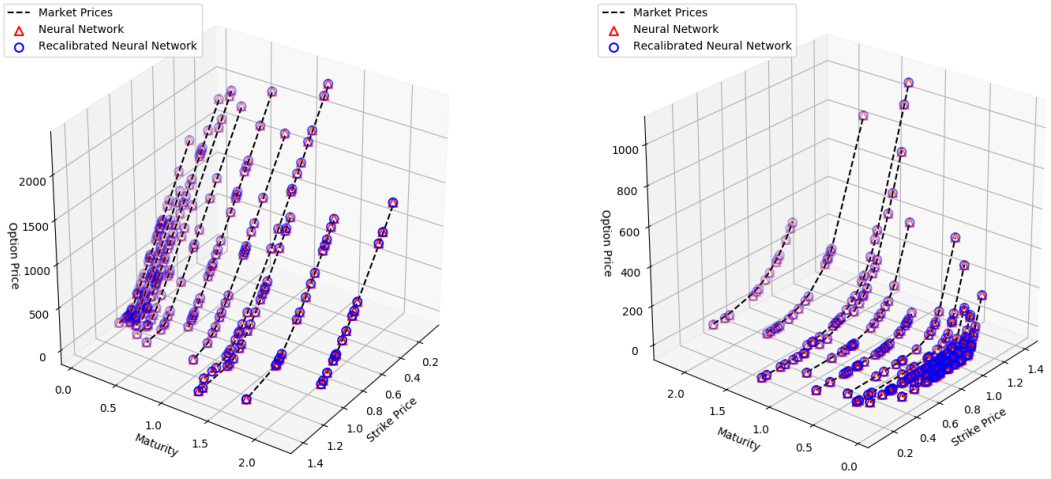


Figure 2.4: We compare the model performance for fitting the out-of-sample option curve on the 20th day (2017-10-02). The notation for different models is the same as figure 2.2.

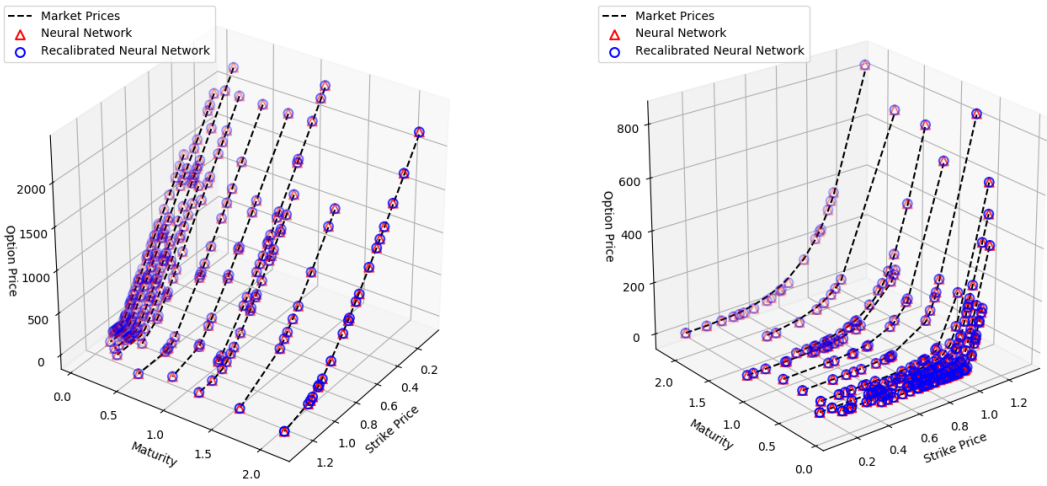


Figure 2.5: We compare the model performance for fitting the out-of-sample option curve on the 40th day (2017-10-30). The notation for different models is the same as figure 2.2.

2.6.6 Out-of-sample Payoff Functions

The neural-network-based models are also valid for the testing data with a different out-of-sample payoff function. In the following experiment, we train only on the call options and test on the corresponding put options. The experiment is conducted on 3 days.

Train on Call Options and Test on Put Options.								
Days	BS		Dupire		Heston		2D-NN	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	11.517	11.374	10.608	9.800	11.476	11.372	3.423	4.062
	417.277	489.360	407.999	334.586	415.468	488.180	44.431	54.009
	56.350%	70.914%	26.218%	68.255%	54.958%	70.908%	10.544%	55.979%
2017-10-23	12.457	11.461	11.393	8.977	12.400	11.454	3.144	2.724
	356.374	421.542	334.358	219.252	356.032	420.266	24.972	18.435
	51.314%	71.344%	27.876%	68.126%	49.688%	71.355%	8.833%	49.297%
2017-11-10	11.351	11.348	10.132	10.230	11.291	11.335	3.977	3.149
	328.135	451.155	306.366	318.385	325.801	448.889	35.787	29.735
	59.638%	69.972%	52.144%	67.729%	57.813%	69.982%	26.553%	51.901%

Table 2.7: Experiment of training on the call options and testing on the put options. "BS" means we use the Black Scholes model to estimate the implied volatility as a scalar with the call options data. There are three columns for each cell. The first column is for "MAE". "MAE" means mean absolute error. The second column is for "MSE". "MSE" means mean squared error. The third column is for "relative MAE", which means the relative mean absolute error.

From the table, we observe that the neural-network-based model has the lowest errors, while the other three traditional pricing models have similar performance.

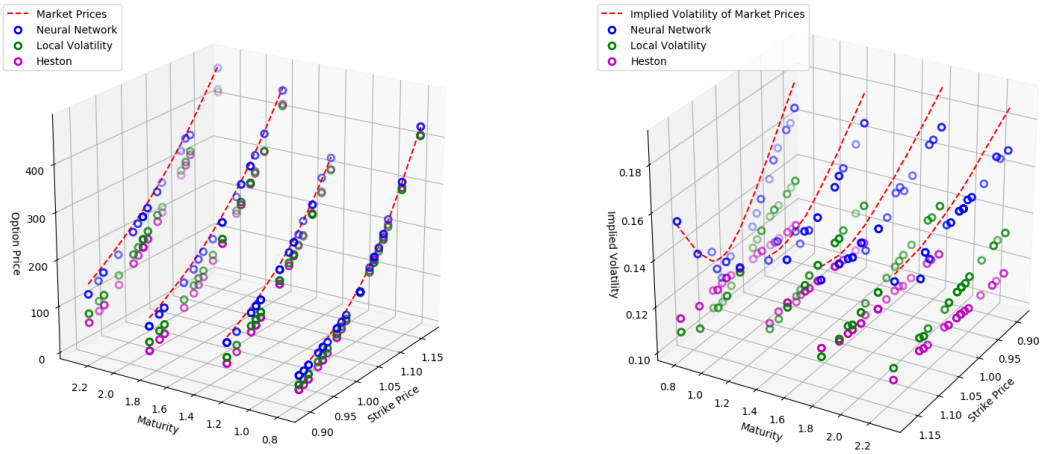


Figure 2.6: Fitting the options curve for 2017-09-01 when we train on the call options and test on the put options. The left figure is the put options curve in which the z-axis is the option price. The right figure is the corresponding implied volatility curve. They are out of sample experiments.

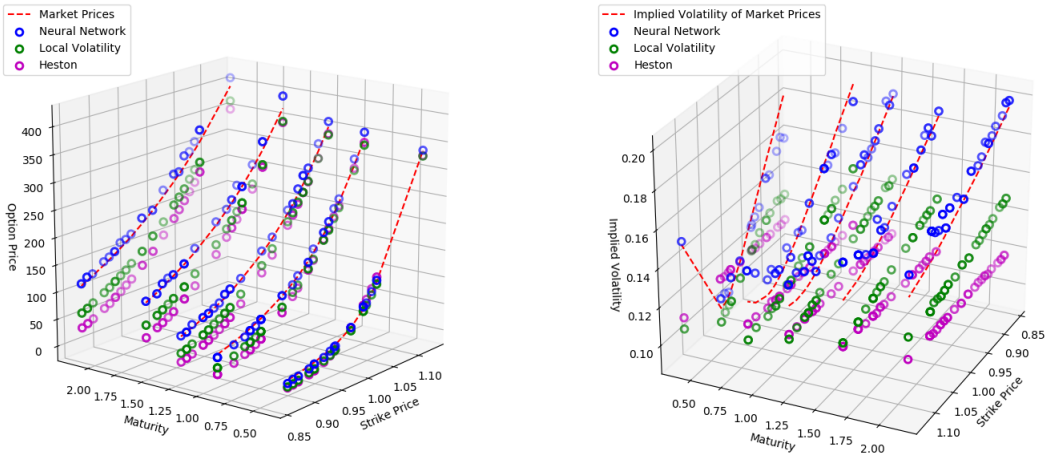


Figure 2.7: Fitting the options curve for 2017-10-23 when we train on the call options and test on the put options. The left figure is the put options curve in which the z-axis is the option price. The right figure is the corresponding implied volatility curve. They are out of sample experiments.

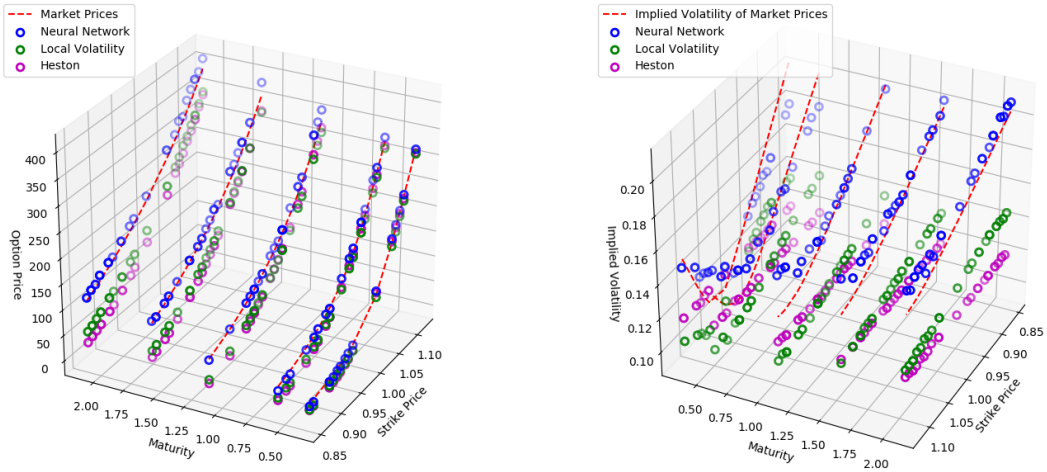


Figure 2.8: Fitting the options curve for 2017-11-10 when we train on the call options and test on the put options. The left figure is the put options curve in which the z-axis is the option price. The right figure is the corresponding implied volatility curve. They are out of sample experiments.

2.6.7 Out-of-sample Strike Prices

We also test the model performance for unseen strike prices. We train the models on the contracts that have strike prices equal or below 2600 and test on the remaining contracts.

Experiment of out-of-sample strike prices.								
Days	BS Scalar		Dupire		Heston		2D-NN	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	14.929	9.540	2.763	16.609	14.663	9.692	3.601	3.418
	433.947	179.440	21.242	645.131	418.509	187.903	41.155	24.706
	647.346%	2.834%	100.818%	4.332%	637.524%	2.865%	99.672%	1.229%
2017-10-23	21.278	11.203	5.846	12.499	20.596	11.163	10.010	11.938
	684.422	190.882	63.031	359.448	642.026	188.292	175.755	233.962
	675.996%	7.887%	145.496%	4.689%	654.255%	7.744%	246.705%	7.281%
2017-11-10	23.299	11.397	5.451	14.337	21.147	10.755	7.855	9.857
	589.513	201.132	53.887	516.926	704.976	179.514	137.897	223.178
	834.499%	8.738%	226.044%	5.952%	749.512%	8.053%	129.564%	4.243%

Table 2.8: Train on options with small strike prices and test on the ones with large strike prices. "BS Scalar" means we use the Black Scholes model to estimate the implied volatility as a scalar. There are three columns for each cell. The first column is for "MAE". "MAE" means mean absolute error. The second column is for "MSE". "MSE" means mean squared error. The third column is for "relative MAE", which means the relative mean absolute error.

Clearly, we can see that the 2D-NN model has the lowest combined errors (sum of the errors for call and put options) among all the models. Also, we can observe that the "relative MAE" for call options is very large. This is because we focus on the options with large strike prices where the call option prices are very small.

The following figures are the fitted call and put option curves.

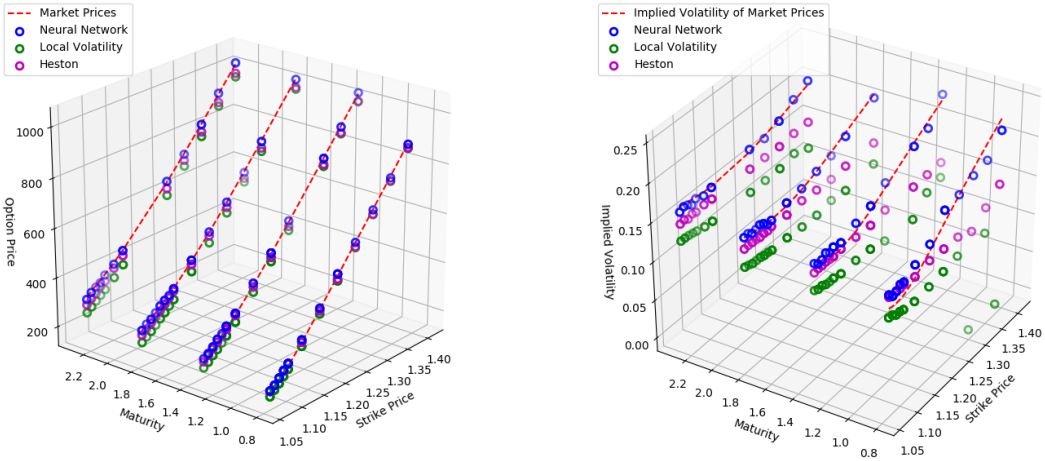


Figure 2.9: Fitting the options curve for 2017-09-01 when we train on the small strike price contracts and test on the contracts with large strike prices. The left figure is the call option curve and the right figure is the put option curve. The z-axis is the option price. They are out of sample experiments.

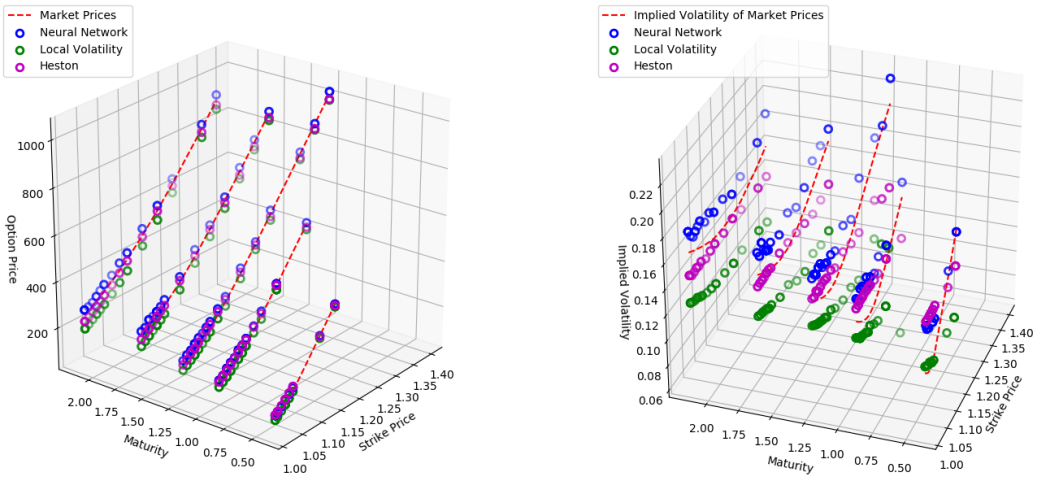


Figure 2.10: Fitting the options curve for 2017-10-23 when we train on the small strike price contracts and test on the contracts with large strike prices. The left figure is the call option curve and the right figure is the put option curve. The z-axis is the option price. They are out of sample experiments.

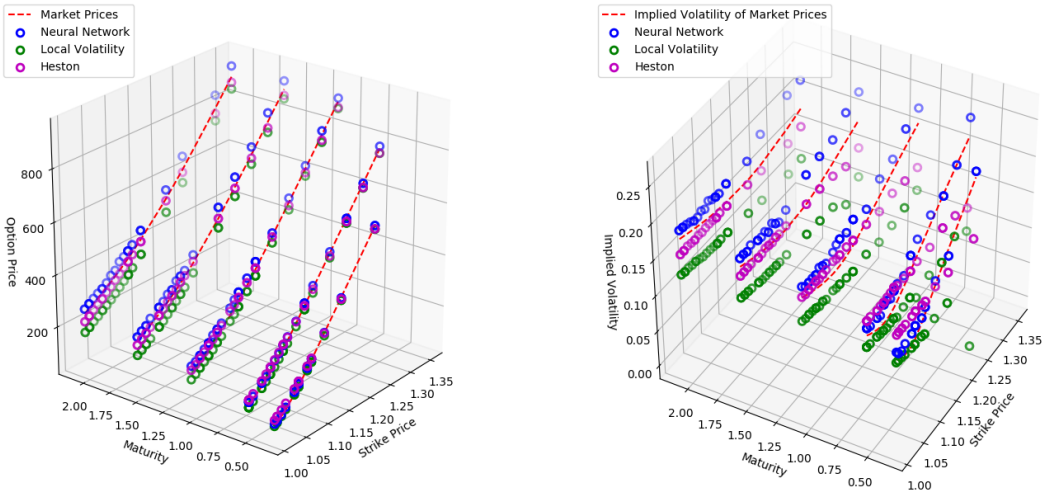


Figure 2.11: Fitting the options curve for 2017-11-10 when we train on the small strike price contracts and test on the contracts with large strike prices. The left figure is the call option curve and the right figure is the put option curve. The z-axis is the option price. They are out of sample experiments.

Chapter 3

Delta Hedging for European Options

3.1 Introduction

The previous chapter of this thesis shows that the neural-network-based models have better performance on options price prediction. In this chapter, we evaluate the performance of our models on delta hedging.

Here the definition of the delta is

$$\Delta = \frac{dP}{dS}$$

where P is the option contract price and S is the underlying stock price.

Specifically in this chapter, we need to compute the following delta hedging errors over a period T:

$$\textbf{Mean Absolute Error} = \frac{1}{N}\sum_{i}^{N} |(\Delta P_i) - (\Delta_i)(\Delta S_i)| \tag{3.1}$$

$$\textbf{Mean Squared Error} = \frac{1}{N}\sum_{i}^{N}((\Delta P_i) - (\Delta_i)(\Delta S_i))^2 \tag{3.2}$$

$$\textbf{Relative Mean Absolute Error} = \frac{1}{N}\sum_{i}^{N}\frac{|(\Delta P_i) - (\Delta_i)(\Delta S_i)|}{P_i} \times 100\% \tag{3.3}$$

where

- ΔP_i **:** the option price change over period T for contract i.
- ΔS_i **:** the stock price change over period T for contract i.
- Δ_i **:** the delta for contract i at the current time.
- N **:** the total number of contracts, including call and put options.

When calculating the above errors, ΔC and ΔS are both from the dataset and are fixed given T. However, the delta Δ is different across different models. Here we use the approximation equation to calculate Δ.

$$\Delta = \frac{f(S_0 + \Delta S) - f(S_0 - \Delta S)}{2\Delta S} \tag{3.4}$$

where $f(S)$ is a function of the contract price with respect to the stock price S, S_0 is the current stock price, and ΔS is a very small change of S. For each model, we simulate $f(S_0 + \Delta S)$ and $f(S_0 - \Delta S)$ simultaneously using the same set of Brownian motions.

3.2 The Next Day Hedging

This section contains the hedging performance for the next day out-of-sample experiment (section 2.6.4). Here we identify two cases for the Black-Scholes model.

- **BS-Scalar:** We estimate one scalar implied volatility for all the training data. It is a single parameter model for all (strike, maturity) pairs.
- **BS-LV:** We estimate the implied volatility for each (strike price, maturity) pair. So there is a vector of implied volatility for the training data.

Below are the results for different models.

Delta Hedging for the Next Day, Part 1.								
Days	BS-LV		BS-Scalar		Dupire		NNLV	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	0.971 1.653 5.533%	0.777 1.183 8.247%	1.258 2.524 24.551%	1.138 2.416 16.520%	1.466 3.374 23.483%	1.226 2.872 17.376%	1.293 3.362 8.376%	1.149 3.058 15.603%
2017-10-23	0.783 0.746 3.104%	0.291 0.177 5.059%	0.833 0.824 5.149%	0.237 0.158 3.444%	0.832 0.841 4.404%	0.247 0.162 3.479%	0.831 0.845 2.895%	0.248 0.160 3.415%
2017-11-10	0.249 0.098 2.080%	0.218 0.108 6.291%	0.274 0.111 4.295%	0.262 0.140 7.023%	0.261 0.107 3.046%	0.248 0.129 7.020%	0.268 0.133 1.457%	0.263 0.168 7.011%
2017-12-07	1.478 2.570 4.506%	0.233 0.145 3.497%	1.757 3.480 15.957%	0.559 0.573 8.923%	1.571 3.027 11.620%	0.536 0.550 8.893%	1.804 4.180 2.582%	0.817 1.243 9.853%
2018-01-26	5.756 37.351 7.447%	0.977 1.936 13.561%	5.897 37.951 11.458%	1.370 3.533 20.261%	5.753 36.694 9.286%	1.441 3.922 20.016%	5.962 40.480 7.041%	1.696 5.394 20.461%

Table 3.1: Delta hedging for the next day, part 1. The three columns in each cell are MAE, MSE, Relative MAE, respectively. "BS" means the Black-Scholes model. "Dupire" means Dupire's model. "NNLV" is the neural-network local volatility model.

Delta Hedging for the Next Day, Part 2.								
Days	SDENN		SDENN-Drift		2D-NN		Heston	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	1.302 3.398 9.318%	1.160 3.100 15.605%	1.194 3.047 9.275%	1.134 2.973 14.907%	1.047 2.500 4.233%	0.988 2.412 14.556%	1.255 2.525 17.940%	0.135 2.403 16.903%
2017-10-23	0.830 0.843 3.063%	0.249 0.161 3.478%	0.799 0.780 3.016%	0.253 0.157 3.529%	0.779 0.745 2.565%	0.264 0.159 3.778%	0.827 0.814 3.938%	0.232 0.153 3.312%
2017-11-10	0.264 0.131 1.569%	0.263 0.168 6.999%	0.260 0.125 1.777%	0.257 0.159 6.889%	0.260 0.116 1.583%	0.241 0.139 6.840%	0.273 0.112 3.649%	0.261 0.141 7.059%
2017-12-07	1.815 4.134 3.416%	0.780 1.146 9.631%	1.687 3.618 2.441%	0.745 1.019 9.416%	1.663 3.436 3.346%	0.672 0.961 7.376%	1.755 3.514 10.174%	0.581 0.615 9.334%
2018-01-26	5.964 40.411 7.401%	1.681 5.290 20.402%	5.780 37.995 6.788%	1.640 5.048 20.176%	5.616 35.617 5.857%	1.483 4.423 18.378%	5.901 38.410 9.292%	1.453 3.975 20.722%

Table 3.2: Delta hedging for the next day, part 2. The three columns in each cell are MAE, MSE, Relative MAE, respectively. "SDENN" refers to the SDE-optimized neural network. "SDENN-Drift" is the SDE-optimized neural network with the drift additionally modeled as another neural network, while "2D-NN" refers to the 2-dimensional neural networks. "Heston" refers to the Heston model.

We can observe that the neural-network-based SDE model outperforms Dupire's local volatility model and the Heston model while slightly under-performing the Black-Scholes model.

3.3 At-The-Money Hedging

In order to make this case simple, we set $T = 1$ (the one-day delta hedging). Furthermore, we select the contracts that satisfy all the conditions below:

- The maturity is between one month and one year.
- Contracts that the strike price is between 2200 and 2800.

- Contracts that the delta exists in the WRDS dataset.

Below are the at-the-money hedging results for different models.

At-the-money Delta Hedging for the Next Day, Part 1.								
Days	BS-LV		BS-Scalar		Dupire		NNLV	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	1.703	1.528	2.254	2.171	2.671	2.533	2.892	2.784
	3.541	2.519	5.403	5.123	7.650	7.129	9.846	9.029
	2.922%	6.471%	10.668%	11.101%	10.040%	12.468%	7.644%	12.400%
2017-10-23	0.683	0.497	0.793	0.409	0.815	0.413	0.822	0.423
	0.627	0.352	0.810	0.293	0.876	0.291	0.916	0.282
	4.512%	3.049%	8.544%	1.878%	7.089%	1.790%	4.204%	1.738%
2017-11-10	0.328	0.373	0.372	0.461	0.338	0.450	0.381	0.496
	0.163	0.212	0.188	0.284	0.173	0.278	0.248	0.372
	2.676%	2.649%	5.946%	3.503%	4.101%	3.558%	2.074%	3.640%
2017-12-07	1.652	0.263	2.074	0.828	1.926	0.774	2.424	1.284
	3.003	0.109	4.621	0.900	4.135	0.823	6.715	2.187
	2.669%	1.376%	5.705%	7.654%	4.716%	7.871%	3.090%	9.804%
2017-01-26	7.708	1.477	7.637	2.392	7.391	2.247	7.899	2.643
	59.894	2.469	58.972	6.217	55.409	5.760	63.437	8.017
	2.849%	10.636%	2.766%	18.083%	2.811%	18.075%	3.007%	18.608%

Table 3.3: At-the-money delta hedging for the next day, part 1. The three columns in each cell are MAE, MSE, Relative MAE, respectively. 'BS' means the Black-Scholes model. 'Dupire' means Dupire's model. 'NNLV' is the neural-network local volatility model.

At-the-money Delta Hedging for the Next Day, Part 2.								
Days	SDENN		SDENN-Drift		2D-NN		Heston	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	2.922 10.001 8.329%	2.821 9.208 12.137%	2.797 9.187 8.460%	2.761 8.834 11.255%	2.482 7.645 5.538%	2.458 7.317 11.443%	2.358 5.826 9.563%	2.274 5.517 12.393%
2017-10-23	0.828 0.924 4.326%	0.424 0.285 1.752%	0.772 0.808 4.282%	0.427 0.275 1.857%	0.748 0.757 3.853%	0.432 0.269 2.003%	0.782 0.785 6.085%	0.395 0.280 1.568%
2017-11-10	0.379 0.249 2.070%	0.497 0.372 3.606%	0.355 0.228 2.148%	0.481 0.351 3.413%	0.347 0.202 2.293%	0.455 0.309 3.403%	0.373 0.191 5.081%	0.463 0.287 3.592%
2017-12-07	2.432 6.601 3.607%	1.229 2.035 9.355%	2.234 5.696 2.832%	1.142 1.728 8.895%	2.260 5.584 4.344%	1.071 1.713 6.392%	2.136 4.869 5.020%	0.862 0.976 8.602%
2017-01-26	7.882 63.140 2.997%	2.620 7.865 18.501%	7.678 59.806 2.926%	2.526 7.314 17.852%	7.358 54.847 2.793%	2.237 5.764 15.803%	7.789 61.274 2.868%	2.543 7.051 19.566%

Table 3.4: At-the-money delta hedging for the next day, part 2 . The three columns in each cell are MAE, MSE, Relative MAE, respectively. "SDENN" refers to the SDE-optimized neural network. "SDENN-Drift" is the SDE-optimized neural network with the drift additionally modeled as another neural network, while "2D-NN" refers to the 2-dimensional neural networks. "Heston" refers to the Heston model.

We can observe that the neural-network-based SDE model underperforms the traditional options pricing models in the case of at-the-money delta hedging.

3.4 Hedging for Out-of-sample Payoff Functions

This section contains the hedging performance for the experiment in section 2.6.6. Although the models are trained only on the call options and we mainly focus on the corresponding put options, we give the hedging results for both call and put options. When designing the experiments, we consider different time periods (one day, two months, and one year).

The following table contains three separate experiments for the next day hedging.

Hedging performance for training on call options and testing on put Options.								
Days	BS-Scalar		Dupire		Heston		2D-NN	
	Call	Put	Call	Put	Call	Put	Call	Put
2017-09-01	0.905	1.094	5.782	2.053	1.095	1.289	0.927	1.000
	1.584	2.810	92.507	15.588	2.201	3.504	2.108	2.446
	6.173%	17.772%	18.058%	17.965%	21.047%	17.664%	3.810%	16.696%
2017-10-23	0.877	0.291	1.940	0.703	0.920	0.279	0.829	0.288
	1.022	0.222	9.423	2.170	1.100	0.214	0.910	0.217
	2.761%	3.457%	5.873%	4.005%	5.502%	3.500%	2.357%	3.533%
2017-11-10	0.350	0.279	0.996	0.458	0.366	0.304	0.386	0.269
	0.348	0.201	2.170	0.638	0.356	0.221	0.430	0.207
	2.651%	7.782%	8.670%	7.695%	5.261%	7.750%	3.250%	7.594%

Table 3.5: Hedging for training on call options and testing on put options. "BS-Scalar" means we use the Black-Scholes model to estimate the implied volatility (scalar) for call options and then use it to calculate the delta for the put options with the same strike prices and maturity. There are three columns for each cell. The first column is for "MAE". "MAE" means mean absolute error. The second column is for "MSE". "MSE" means mean squared error. The third column is for "relative MAE", which means the relative mean absolute error.

We can observe the neural-network-based SDE model outperforms Dupire's local volatility model and the Heston model, but slightly under-performs the Black-Scholes model.

Hedging with Recalibration

In the next experiment, we explore the hedging performance of put options for two months. We have four models below.

- For the Black-Scholes model, we use the implied volatility from the call option for hedging the corresponding put option (with the same K and T). Specifically, we have option pairs $C^i_{K,T}$ and $P^i_{K,T}$, where K, T are strike prices and maturity. i means the i-th day. We use the Black-Scholes formula to get the implied volatility $\sigma^i_{K,T}$ from the $C^i_{K,T}$. Then we use $\sigma^i_{K,T}$ to calculate the delta for $P^i_{K,T}$ using Black-Scholes formula again.

- For the Dupire's local volatility model, we use all the call options for each day for cubic spline interpolation. So this should be a re-calibrated method.
- For 2D-NN and the Heston model, similarly, the models are trained on call options to get the parameters for hedging.

Additionally, we consider the cases with and without recalibration (2.6.5). Here we don't have results for the Black-Scholes model without recalibration since the implied volatility for unseen strike price and maturity cannot be calculated.

Hedging for two months period (without recalibration).								
Days	BS		Dupire		Heston		2D-NN	
	Call	Put	Call	Put	Call	Put	Call	Put
2017/09 ∼ 2017/10	NA	NA	2.466	0.870	1.044	0.486	0.963	0.449
	NA	NA	19.332	4.171	2.138	0.674	1.875	0.615
	NA	NA	7.698%	7.344 %	7.224%	6.982%	2.348%	6.650%

Table 3.6: Hedging for out-of-sample payoff functions over two months period without recalibration. There are three columns for each cell. The first column is for "MAE". "MAE" means mean absolute error. The second column is for "MSE". "MSE" means mean squared error. The third column is for "relative MAE", which means the relative mean absolute error.

Hedging for two months period (with recalibration).								
Days	BS		Dupire		Heston		2D-NN	
	Call	Put	Call	Put	Call	Put	Call	Put
2017/09 ∼ 2017/10	0.986	0.473	2.813	0.950	1.043	0.487	0.947	0.428
	1.999	0.635	31.715	5.811	2.139	0.675	1.781	0.557
	2.887%	6.967%	7.696%	7.540%	6.823%	6.995%	2.876%	6.747%

Table 3.7: Hedging for out-of-sample payoff functions over two months period with recalibration. The evaluation metrics are the same as the previous table.

Based on the results in the tables, we can observe that the 2D-NN models still have the best performance overall and the recalibration method helps reduce its hedging errors.

Chapter 4

Neural-network PDEs for American Options

Option prices can also be obtained with the PDE method where the option prices follow the backward Kolmogorov equation [24]. While the SDE method is only computationally efficient on European options, the PDE model allows for calibration using a much wider class of objective functions and can be used for American options.

The option price for the American option is

$$P_t^{American} = \sup_{\textit{All possible strategies for selecting } \tau \in [t,T]} \mathbb{E}\left[e^{-r(\tau-t)} f(S_\tau)\right] \tag{4.1}$$

where $P_t^{American}$ is the option price and $f()$ is the payoff function.

Consider the price of an asset at time t, S_t, that follows a geometric Brownian Motion (GBM) with the following stochastic differential equation (SDE):

$$dS_t = \mu S_t dt + \sigma(S_t, t) S_t dW_t \tag{4.2}$$

Define the option price u is a function of time t and the underlying asset price S_t.

$$u(t,s) = \mathbb{E}[e^{-r(T-t)} f(S_T)|S_t = s] \tag{4.3}$$

Then $u(t,s)$ satisfies the backward Kolmogorov equation [24]:

$$-\frac{\partial u}{\partial t} = rS\frac{\partial u}{\partial S} + \frac{1}{2}(\sigma(S,t)S)^2\frac{\partial^2 u}{\partial S^2} - ru \tag{4.4}$$

with the condition $u(T,s) = f(s)$.

Using the coordinate transformation $t \to (T-t)$, we can transform this into the PDE with an initial condition:

$$\frac{\partial u}{\partial t} = rS\frac{\partial u}{\partial S} + \frac{1}{2}(\sigma(S,t)S)^2\frac{\partial^2 u}{\partial S^2} - ru \tag{4.5}$$

with the condition $u(0,s) = f(s)$. The objective function is to minimize:

$$L(\sigma) = (u(T, S_0; \sigma) - P_{market})^2 \tag{4.6}$$

We can numerically solve equation (4.5) using finite difference method. Here we use an intermediate variable $v_{n+1,j}$ to help determine whether we execute the option at each time step:

$$v_{n+1,j} = u_{n,j} + (rS_j\frac{u_{n,j+1} - u_{n,j-1}}{2\Delta_S} + \frac{1}{2}\sigma^2 S_j^2\frac{u_{n,j+1} - 2u_{n,j} + u_{n,j-1}}{\Delta_S^2} - ru_{n,j})\Delta_t \tag{4.7}$$

$$u_{n+1,j} = \max(v_{n+1,j}, h_j) \tag{4.8}$$

where $h_j = \max(S_j - K, 0)$ for a call option and $h_j = \max(K - S_j, 0)$ for a put option.

For any $i \in [0, N]$ and $j \in [0, L]$, we have

$$u_{0,j} = f(S_j) \tag{4.9}$$

$$u_{i,0} = 0 \tag{4.10}$$

$$u_{i,L} = e^{-ri\Delta_t}(S_L - K)^+ \tag{4.11}$$

The corresponding objective function is to minimize:

$$L(\sigma) = (u_{N,I_0} - P_{market})^2 \tag{4.12}$$

where $N = T/\Delta_t$ and I_0 means the index of the initial stock value in the grid.

Pytorch can calculate the chain rule in backpropagation using automatic differentiation. The whole optimization algorithm is below:

Algorithm 3 Optimization algorithm for the backward Kolmogorov equation

procedure
  $S \leftarrow ones(N_S)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i * \Delta_S$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow ones(N+1, N_S)$
    $u[0,:] \leftarrow max(S-K, 0)$
    for $i \leftarrow 1$ `to` N **do**
      $u[i,0] \leftarrow 0$
      $u[i,-1] \leftarrow S[-1] - K$
      $u_s \leftarrow (u[i,2:] - u[i,:-2])/(2\Delta_S)$
      $u_{ss} \leftarrow (u[i,2:] - 2u[i,1:-1] + u[i,:-2])/(\Delta_S^2)$
      $\sigma \leftarrow g(S,t)$
      $L \leftarrow rx[1:-1,0]u_s + \frac{1}{2}(\sigma[1:-1]S[1:-1])^2 u_{ss} - ru[i,1:-1]$
      $u[i+1,1:-1] \leftarrow u[i,1:-1] + \Delta_t L$
    end for
    $loss \leftarrow (u[-1,I_0] - P_{market})^2$
    loss.backward()
    optimizer.step()
  end for
end procedure

where $g(S,t)$ is a function for the volatility. I_0 means the index of the initial stock value in the grid.

4.1 The Black-Scholes PDE model

One of the assumptions for the Black-Scholes model is constant drift and diffusion (implied volatility). So the corresponding PDE for the Black-Scholes model is:

$$\frac{\partial u}{\partial t} = (r-d)S\frac{\partial u}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 u}{\partial S^2} - ru \tag{4.13}$$

where r is the risk-free interest rate and d is the dividend rate. Both of them are scalars.

We will use the previously described finite difference method to solve the problem. Below is the algorithm.

Algorithm 4 Optimization algorithm for Black-Scholes PDE

procedure
  $S \leftarrow ones(N_S)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i * \Delta_S$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow ones(N+1, N_S)$
    $u[0,:] \leftarrow max(S-K, 0)$
    for $i \leftarrow 1$ `to` N **do**
      $u[i,0] \leftarrow 0$
      $u[i,-1] \leftarrow S[-1] - K$
      $u_s \leftarrow (u[i,2:] - u[i,:-2])/(2 * \Delta_S)$
      $u_{ss} \leftarrow (u[i,2:] - 2 * u[i,1:-1] + u[i,:-2])/(\Delta_S^2)$
      $L \leftarrow rx[1:-1,0]u_s + \frac{1}{2}(\sigma S[1:-1])^2 u_{ss} - ru[i,1:-1]$
      $u[i+1,1:-1] \leftarrow u[i,1:-1] + \Delta_t L$
    end for
    $loss \leftarrow (u[-1,I_0] - P_{market})^2$
    loss.backward()
    optimizer.step()
  end for
end procedure

where σ is the variable we should estimate.

4.2 1-Dimensional Neural-network PDE model

The PDE for the 1-dimensional neural-network model (1D-NN) is

$$\frac{\partial u}{\partial t} = (r-d)S\frac{\partial u}{\partial S} + \frac{1}{2}(g(S)S)^2\frac{\partial^2 u}{\partial S^2} - ru \tag{4.14}$$

where $g(S)$ is a neural network with current price S. It replaces the constant drift σ in equation (4.13). Everything else is the same as the explicit method.

Algorithm 5 Optimization algorithm for 1-dimensional neural network PDE

procedure
  $S \leftarrow ones(N_S)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i\Delta_S$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow ones(N+1, N_S)$
    $u[0,:] \leftarrow max(S-K, 0)$
    for $i \leftarrow 1$ `to` N **do**
      $u[i,0] \leftarrow 0$
      $u[i,-1] \leftarrow S[-1] - K$
      $u_s \leftarrow (u[i,2:] - u[i,:-2])/(2 * \Delta_S)$
      $u_{ss} \leftarrow (u[i,2:] - 2u[i,1:-1] + u[i,:-2])/(\Delta_S^2)$
      $\sigma \leftarrow g(S)$
      $L \leftarrow rx[1:-1,0]u_s + \frac{1}{2}(\sigma[1:-1]S[1:-1])^2 u_{ss} - ru[i,1:-1]$
      $u[i+1,1:-1] \leftarrow u[i,1:-1] + \Delta_t L$
    end for
    $loss \leftarrow (u[-1,I_0] - P_{market})^2$
    loss.backward()
    optimizer.step()
  end for
end procedure

where $g(S)$ is a neural network for the volatility.

4.3 Neural-network Local Volatility PDE model

The PDE for the neural-network local volatility model (NNLV) is

$$\frac{\partial u}{\partial t} = (r-d)S\frac{\partial u}{\partial S} + \frac{1}{2}(g(S,t)S)^2\frac{\partial^2 u}{\partial S^2} - ru \tag{4.15}$$

where $g(S,t)$ is a neural network with inputs price S and time t. It replaces the constant drift σ in equation (4.13). Everything else is the same as the explicit method.

Algorithm 6 Optimization algorithm for neural-network local volatility PDE

procedure
  $S \leftarrow ones(N_S)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i\Delta_S$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow ones(N+1, N_S)$
    $u[0,:] \leftarrow max(S-K, 0)$
    for $i \leftarrow 1$ `to` N **do**
      $u[i,0] \leftarrow 0$
      $u[i,-1] \leftarrow S[-1] - K$
      $u_s \leftarrow (u[i,2:] - u[i,:-2])/(2 * \Delta_S)$
      $u_{ss} \leftarrow (u[i,2:] - 2u[i,1:-1] + u[i,:-2])/(\Delta_S^2)$
      $\sigma \leftarrow g(S,t)$
      $L \leftarrow rx[1:-1,0]u_s + \frac{1}{2}(\sigma[1:-1]S[1:-1])^2 u_{ss} - ru[i,1:-1]$
      $u[i+1,1:-1] \leftarrow u[i,1:-1] + \Delta_t L$
      $h_0 \leftarrow max(S-K, 0)$
      $u \leftarrow max(u, h_0)$
    end for
    $loss \leftarrow (u[-1, I_0] - P_{market})^2$
    loss.backward()
    optimizer.step()
  end for
end procedure

where $g(S,t)$ is a neural network for the volatility.

4.4 Stochastic Volatility PDE models

4.4.1 Explicit Method For the Heston Model

The mathematical equations for Heston model is below:

$$dS_t = \mu S_t dt + \sqrt{Y_t} S_t dW_t^S \tag{4.16}$$

$$dY_t = \alpha(m - Y_t)dt + k\sqrt{Y_t} dW_t^Y \tag{4.17}$$

where W_t^S and W_t^Y are Wiener processes with correlation ρ. Here μ is the drift, Y_0, α, m, k, ρ are parameters for the Heston model. More specifically:

- Y_0 is the initial value for Y_t.
- m is the long variance. As t tends to infinity, the expected value of Y_t tends to m.
- α is the rate at which Y_t reverts to m.
- k is the volatility of the volatility and determines the variance of Y_t.

We correspondingly estimate Y_0, α, m, k, ρ with the constrains that Y_0, α, m, k are nonnegative. Moreover, if the parameters satisfy the Feller condition $2\alpha m > k^2$, the process Y_t is strictly positive.

The corresponding Heston PDE is

$$\frac{\partial u}{\partial t} = \frac{1}{2} Y S^2 \frac{\partial^2 u}{\partial S^2} + \rho k Y S \frac{\partial^2 u}{\partial Y \partial S} + \frac{1}{2} k^2 Y \frac{\partial^2 u}{\partial Y^2} - ru + (r-q) S \frac{\partial u}{\partial S} + \alpha(m - Y) \frac{\partial u}{\partial Y} \tag{4.18}$$

The finite difference method and boundary conditions are referred from this book [34]. Using finite difference method (n, i, j for time, S grid, Y grid respectively), we have

$$\frac{\partial u_{n,i,j}}{\partial S_i} = \frac{u_{n,i+1,j} - u_{n,i-1,j}}{2\Delta_S} \tag{4.19}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial S_i^2} = \frac{u_{n,i+1,j} - 2u_{n,i,j} + u_{n,i-1,j}}{\Delta_S^2} \tag{4.20}$$

$$\frac{\partial u_{n,i,j}}{\partial Y_j} = \frac{u_{n,i,j+1} - u_{n,i,j-1}}{2\Delta_Y} \tag{4.21}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial Y_j^2} = \frac{u_{n,i,j+1} - 2u_{n,i,j} + u_{n,i,j-1}}{\Delta_Y^2} \tag{4.22}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial S_i \partial Y_j} = \frac{u_{n,i+1,j+1} - u_{n,i-1,j+1} - u_{n,i+1,j-1} + u_{n,i-1,j-1}}{4\Delta_S \Delta_Y} \tag{4.23}$$

So the final update equation is

$$\begin{aligned} u_{n+1,i,j} = u_{n,i,j} + \Delta_t \Big(& \frac{1}{2} Y_j S_i^2 \frac{u_{n,i+1,j} - 2u_{n,i,j} + 2u_{n,i-1,j}}{\Delta_S^2} \\ & + \rho k Y_j S_i \frac{u_{n,i+1,j+1} - u_{n,i-1,j+1} - u_{n,i+1,j-1} + u_{n,i-1,j-1}}{4\Delta_S \Delta_Y} \\ & + \frac{1}{2} k^2 Y_j \frac{u_{n,i,j+1} - 2u_{n,i,j} + 2u_{n,i,j-1}}{\Delta_Y^2} - r u_{n,i,j} \\ & + (r-q) S_i \frac{u_{n,i+1,j} - u_{n,i-1,j}}{2\Delta_S} + \alpha (m - Y) \frac{u_{n,i,j+1} - u_{n,i,j-1}}{2\Delta_Y} \Big) \end{aligned} \tag{4.24}$$

The corresponding boundary conditions in $u(n, i, j)$ ($n \in [0, N_t]$, $i \in [0, N_S]$, $j \in [0, N_Y]$) are

- Boundary Condition for t=0:

$$u[0, i, :] = \max(S_i - K, 0), \quad \forall i \in [0, N_S] \tag{4.25}$$

- Boundary Condition for $S = S_{min} = 0$:

$$u[:, 0, 0 : N_Y] = f(0) = 0, \quad \forall n \in [0, N_t], \quad \forall j \in [0, N_Y] \tag{4.26}$$

- Boundary Condition for $S = S_{max}$:

$$u[n, N_S, 0 : N_Y] = \max(S_{N_S} - K, 0), \quad \forall n \in [0, N_t] \tag{4.27}$$

- Boundary Condition for $Y = Y_{max}$:

  We have $\frac{\partial^2 u}{\partial Y^2} = 0$. We apply it to the equation (4.18)

$$\frac{\partial u}{\partial t} = \frac{1}{2} Y S^2 \frac{\partial^2 u}{\partial S^2} + \rho k Y S \frac{\partial^2 u}{\partial Y \partial S} - ru + (r - q) S \frac{\partial u}{\partial S} + \alpha(m - Y) \frac{\partial u}{\partial Y} \tag{4.28}$$

  Here we use central differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,i,-1}}{\partial S} \approx \frac{u_{n,i+1,-1} - u_{n,i-1,-1}}{2\Delta_S} \tag{4.29}$$

$$\frac{\partial^2 u_{n,i,-1}}{\partial S^2} \approx \frac{u_{n,i+1,-1} - 2u_{n,i,-1} + u_{n,i-1,-1}}{\Delta_S^2} \tag{4.30}$$

  We use forward differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,i,-1}}{\partial Y} \approx \frac{u_{n,i,-1} - u_{n,i,-2}}{\Delta_Y} \tag{4.31}$$

  For $\frac{\partial^2 u}{\partial S \partial Y}$ we combine central differences and forward differences.

$$\begin{aligned}
\frac{\partial^2 u_{n,i,-1}}{\partial S \partial Y} &\approx \frac{\partial}{\partial S} \frac{u_{n,i,-1} - u_{n,i,-2}}{\Delta_Y} && (4.32)\\
&= \frac{1}{\Delta_Y} \left(\frac{\partial u_{n,i,-1}}{\partial S} - \frac{\partial u_{n,i,-2}}{\partial S} \right) && (4.33)\\
&\approx \frac{1}{\Delta_Y} \left(\frac{u_{n,i+1,-1} - u_{n,i-1,-1}}{2\Delta_S} - \frac{u_{n,i+1,-2} - u_{n,i-1,-2}}{2\Delta_S} \right) && (4.34)\\
&= \frac{u_{n,i+1,-1} - u_{n,i-1,-1} - u_{n,i+1,-2} + u_{n,i-1,-2}}{2\Delta_S \Delta_Y} && (4.35)
\end{aligned}$$

- Boundary Condition for $Y = Y_{min} = 0$:

  The PDE reduces to

$$\frac{\partial u}{\partial t} = -ru + (r-q)S\frac{\partial u}{\partial S} + \alpha m \frac{\partial u}{\partial Y} \tag{4.36}$$

  Here we use central differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,i,0}}{\partial S} = \frac{u[n,i+1,0] - u[n,i-1,0]}{2\Delta_S} \tag{4.37}$$

  We use forward differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,i,0}}{\partial Y} = \frac{u[n,i,1] - u[n,i,0]}{\Delta_Y} \tag{4.38}$$

Proof of the Boundary Condition

Let's prove the following boundary condition:

$$\frac{\partial^2 u_{n,i,-1}}{\partial S \partial Y} \approx \frac{u_{n,i+1,-1} - u_{n,i-1,-1} - u_{n,i+1,-2} + u_{n,i-1,-2}}{2\Delta_S \Delta_Y} \tag{4.39}$$

With Taylor expansions, $u_{n,i+1,-1}$, $u_{n,i-1,-1}$ and $u_{n,i+1,-2}$ can be rewrite as follows:

$$\begin{aligned} u_{n,i+1,-1} \approx u_{n,i-1,-2} &+ \frac{\partial u_{n,i-1,-2}}{\partial S} 2\Delta_S + \frac{\partial u_{n,i-1,-2}}{\partial Y}\Delta_Y \\ &+ \frac{1}{2}\left(\frac{\partial^2 u_{n,i-1,-2}}{\partial S^2} 4\Delta_S^2 + 2\frac{\partial^2 u_{n,i-1,-2}}{\partial S \partial Y} 2\Delta_S \Delta_Y + \frac{\partial^2 u_{n,i-1,-2}}{\partial Y^2}\Delta_Y^2\right) \\ + \frac{1}{6}\left(\frac{\partial^3 u_{n,i-1,-2}}{\partial S^3} 8\Delta_S^3 + 3\frac{\partial^3 u_{n,i-1,-2}}{\partial S^2 \partial Y} 4\Delta_S^2 \Delta_Y + 3\frac{\partial^3 u_{n,i-1,-2}}{\partial S \partial Y^2} 2\Delta_S \Delta_Y^2 + \frac{\partial^3 u_{n,i-1,-2}}{\partial Y^3}\Delta_Y^3\right) \end{aligned} \tag{4.40}$$

$$u_{n,i-1,-1} \approx u_{n,i-1,-2} + \frac{\partial u_{n,i-1,-2}}{\partial Y}\Delta_Y + \frac{1}{2}\frac{\partial^2 u_{n,i-1,-2}}{\partial Y^2}\Delta_Y^2 + \frac{1}{6}\frac{\partial^3 u_{n,i-1,-2}}{\partial Y^3}\Delta_Y^3 \tag{4.41}$$

$$u_{n,i+1,-2} \approx u_{n,i-1,-2} + \frac{\partial u_{n,i-1,-2}}{\partial S}2\Delta_S + \frac{1}{2}\frac{\partial^2 u_{n,i-1,-2}}{\partial S^2}4\Delta_S^2 + \frac{1}{6}\frac{\partial^3 u_{n,i-1,-2}}{\partial S^3}8\Delta_S^3 \tag{4.42}$$

We apply these equations to the right side of equation (4.39), we have

$$\begin{aligned}\frac{u_{n,i+1,-1} - u_{n,i-1,-1} - u_{n,i+1,-2} + u_{n,i-1,-2}}{2\Delta_S\Delta_Y} \approx \frac{\partial^2 u_{n,i-1,-2}}{\partial S \partial Y} + (\frac{\partial^3 u_{n,i-1,-2}}{\partial S^2 \partial Y}\Delta_S + \\ \frac{\partial^3 u_{n,i-1,-2}}{\partial S \partial Y^2}\frac{1}{2}\Delta_Y)\end{aligned} \tag{4.43}$$

So taking the limit of $\Delta_S \to 0$ and $\Delta_Y \to 0$, we get equation (4.39).

The following is the algorithm for the Heston PDE model.

Algorithm 7 Optimization algorithm for Heston PDE

procedure
  $S \leftarrow ones(N_S)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i\Delta_S$
  end for
  for $j \leftarrow 1$ `to` N_Y **do**
    $Y[j] = j\Delta_Y$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow zeros(N+1, N_S+1, N_Y+1)$
    $u[0, :, :] \leftarrow max(S-K, 0)$
    for $t \leftarrow 1$ `to` N **do**
      $u[t, 0, :-1] \leftarrow 0$
      $u[t, -1, :-1] \leftarrow S[-1] - K$
      $u_y \leftarrow (u[t, 1:-1, -1] - u[t, 1:-1, -2])/(\Delta_Y)$
      $u_s \leftarrow (u[t, 2:, -1] - u[t, :-2, -1])/(2\Delta_S)$
      $u_{ss} \leftarrow (u[t, 2:, -1] - 2u[t, 1:-1, -1] + u[t, :-2, -1])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, 2:, -1] - u[t, :-2, -1] - u[t, 2:, -2] + u[t, :-2, -2])/(2\Delta_S\Delta_Y)$
      $L_1 \leftarrow \frac{1}{2}Y[-1](S[1:-1])^2 u_{ss} + \rho k Y[-1]S[1:-1]u_{sy} - ru[t, 1:-1, 1:-1] + (r-q)S[1:-1]u_s + \alpha(m - Y[-1])u_y$
      $u[t, 1:-1, -1] \leftarrow L_1\Delta_t + u[t, 1:-1, -1]$
      $u_y \leftarrow (u[t, 1:-1, 1] - u[t, 1:-1, 0])/(\Delta_Y)$
      $u_s \leftarrow (u[t, 2:, 0] - u[t, :-2, 0])/(2\Delta_S)$
      $L_2 \leftarrow -ru[t, 1:-1, 0] + (r-q)S[1:-1]u_s + \alpha m u_y$
      $u[t, 1:-1, 0] \leftarrow L_2\Delta_t + u[t, 1:-1, 0]$
      $u_y \leftarrow (u[t, 1:-1, 2:] - u[t, 1:-1, :-2])/(2\Delta_Y)$
      $u_s \leftarrow (u[t, 2:, 1:-1] - u[t, :-2, 1:-1])/(2\Delta_S)$
      $u_{yy} \leftarrow (u[t, 1:-1, 2:] - 2u[t, 1:-1, 1:-1] + u[t, 1:-1, :-2])/(\Delta_Y^2)$
      $u_{ss} \leftarrow (u[t, 2:, 1:-1] - 2u[t, 1:-1, 1:-1] + u[t, :-2, 1:-1])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, 2:, 2:] - u[t, :-2, 2:] - u[t, 2:, :-2] + u[t, :-2, :-2])/(4\Delta_S\Delta_Y)$
      $L_3 \leftarrow \frac{1}{2}Y[1:-1](S[1:-1])^2 u_{ss} + \rho k Y[1:-1]S[1:-1]u_{sy} + \frac{1}{2}k^2 Y[1:-1]u_{yy} - ru[t, 1:-1, 1:-1] + (r-q)S[1:-1]u_s + \alpha(m - Y[1:-1])u_y$
      $u[t, 1:-1, 1:-1] \leftarrow L_3\Delta_t + u[t, 1:-1, 1:-1]$
      $h_0 \leftarrow max(S-K, 0)$
      $u \leftarrow max(u, h_0)$
    end for
    $P \leftarrow interpolate(u[-1, I_0, :], Y0)$
    $loss \leftarrow (P - P_{market})^2$
    loss.backward()
    optimizer.step()
  end for
end procedure

Here in line 37 there is a interpolate function. It means we need to interpolate along the Y direction to get an estimate of the initial value of Y, which is Y_0. This is another parameter we need to train together with other variables.

4.4.2 2-Dimensional Neural-network with Heston Model

We use a neural network $g(S_t, Y_t, t)$ with an output dimension of 2 (g_1 and g_2) to replace the drift and diffusion of the price SDE in the Heston model. We call this model 2DNN-Hestion.

The equations for the 2-dimensional SDEs are

$$dS_t = g_1(t, S_t, Y_t)S_t dt + g_2(t, S_t, Y_t)S_t dW_t^s \tag{4.44}$$

$$dY_t = \alpha(m - Y_t)dt + k\sqrt{Y_t}dW_t^y \tag{4.45}$$

$$dW_t^S dW_t^Y = \rho dt \tag{4.46}$$

Consider the option price with a discounted payoff

$$u(t, s, y) = \mathbb{E}\left[e^{-r(T-t)}f(S_T, Y_T)|S_t = s, Y_t = y\right] \tag{4.47}$$

With tower property, we have

$$
\begin{aligned}
u(t, S_t, Y_t) &= \mathbb{E}\left[e^{-r(T-t)} f(S_T, Y_T)|S_t, Y_t\right] \\
&= \mathbb{E}\left[e^{-rdt}\mathbb{E}\left(e^{-r(T-t-dt)} f(S_T, Y_T)|S_{t+dt}, Y_{t+dt}\right)|S_t, Y_t\right] \\
&= \mathbb{E}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})|S_t, Y_t\right] \\
&= \mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})\right]
\end{aligned} \tag{4.48}
$$

Here using the Taylor series, we have

$$
\begin{aligned}
\mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})\right] &= \mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_t+dS, Y_t+dY)\right] \\
&\approx \mathbb{E}_{S_t,Y_t}[u(t, S_t, Y_t) + (e^{-rdt}(-r)u(t, S_t, Y_t) + e^{-rdt}\partial_t u(t, S_t, Y_t))dt \\
&+ e^{-rdt}(\partial_s u(t, S_t, Y_t)dX + \partial_y u(t, S_t, Y_t)dY + \frac{1}{2}\partial_{ss} u(t, S_t, Y_t)[dS]^2 \\
&+ \frac{1}{2}\partial_{yy} u(t, S_t, Y_t)[dY]^2 + \partial_{sy} u(t, S_t, Y_t)[dSdY])]
\end{aligned} \tag{4.49}
$$

Here we have $\mathbb{E}(dS) = g_1 S dt$, $\mathbb{E}(dY) = \alpha(m - Y)dt$, $\mathbb{E}([dS]^2) = g_2^2 S^2 dt$, $\mathbb{E}([dY]^2) = k^2 Y dt$ and $\mathbb{E}([dSdY]) = \rho g_2 S k\sqrt{Y} dt$. So we have

$$
\begin{aligned}
\mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})\right] &\approx u(t, S_t, Y_t) + e^{-rdt}(-ru + \partial_t u + g_1 S\partial_s u \\
&+ \alpha(m - Y)\partial_y u + \frac{1}{2} g_2^2 S^2 \partial_{ss} u + \frac{1}{2} k^2 Y \partial_{yy} u + \rho g_2 S k\sqrt{Y}\partial_{sy} u)dt
\end{aligned} \tag{4.50}
$$

Compare this with equation (4.48), we have

$$
\begin{aligned}
u(t, S_t, Y_t) \approx u(t, S_t, Y_t) &+ e^{-rdt}(-ru + \partial_t u + g_1 S\partial_s u + \alpha(m - Y)\partial_y u \\
&+ \frac{1}{2} g_2^2 S^2 \partial_{ss} u + \frac{1}{2} k^2 Y \partial_{yy} u + \rho g_2 S k\sqrt{Y}\partial_{sy} u)dt
\end{aligned} \tag{4.51}
$$

Thus, we have

$$-\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2\frac{\partial^2 u}{\partial S^2} + \alpha(m-Y)\frac{\partial u}{\partial Y} + \frac{1}{2}k^2 Y\frac{\partial^2 u}{\partial Y^2} - ru \\ +\rho g_2(s,y,t)Sk\sqrt{Y}\frac{\partial^2 u}{\partial S\partial Y} \tag{4.52}$$

with $u(0,s,y) = \mathbb{E}\left[e^{-rT}f(S_T,Y_T)|S_0 = s, Y_0 = y\right]$ and $u(T,s,y) = f(s,y)$.

In addition, we use the coordinate transformation $t \to (T-t)$, then the PDE becomes

$$\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2\frac{\partial^2 u}{\partial S^2} + \alpha(m-Y)\frac{\partial u}{\partial Y} + \frac{1}{2}k^2 Y\frac{\partial^2 u}{\partial Y^2} - ru \\ +\rho g_2(s,y,t)Sk\sqrt{Y}\frac{\partial^2 u}{\partial S\partial Y} \tag{4.53}$$

with $u(T,s,y) = \mathbb{E}\left[e^{-rT}f(S_T,Y_T)|S_0 = s, Y_0 = y\right]$ and $u(0,s,y) = f(s,y)$.

Using the finite difference method, we have

$$\frac{\partial u_{n,i,j}}{\partial S} = \frac{u_{n,i+1,j} - u_{n,i-1,j}}{2\Delta_S} \tag{4.54}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial S^2} = \frac{u_{n,i+1,j} - 2u_{n,i,j} + u_{n,i-1,j}}{\Delta_S^2} \tag{4.55}$$

$$\frac{\partial u_{n,i,j}}{\partial Y} = \frac{u_{n,i,j+1} - u_{n,i,j-1}}{2\Delta_Y} \tag{4.56}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial Y^2} = \frac{u_{n,i,j+1} - 2u_{n,i,j} + u_{n,i,j-1}}{\Delta_Y^2} \tag{4.57}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial S\partial Y} = \frac{u_{n,i+1,j+1} + u_{n,i-1,j-1} - u_{n,i-1,j+1} - u_{n,i+1,j-1}}{4\Delta_S\Delta_Y} \tag{4.58}$$

So the final update equation is

$$\begin{aligned} u_{n+1,i,j} = u_{n,i,j} + \Delta_t(g_1(s,y,t)S_i\frac{u_{n,i+1,j}-u_{n,i-1,j}}{2\Delta_S} \\ +\frac{1}{2}(g_2(s,y,t)S_i)^2\frac{u_{n,i+1,j}-2u_{n,i,j}+u_{n,i-1,j}}{\Delta_S^2}+\alpha(m-Y_j)\frac{u_{n,i,j+1}-u_{n,i,j-1}}{2\Delta_Y} \\ +\frac{1}{2}k^2Y_j\frac{u_{n,i,j+1}-2u_{n,i,j}+u_{n,i,j-1}}{\Delta_Y^2}-ru_{n,i,j} \\ +\rho g_2(s,y,t)S_ik\sqrt{Y_j}\frac{u_{n,i+1,j+1}+u_{n,i-1,j-1}-u_{n,i-1,j+1}-u_{n,i+1,j-1}}{4\Delta_S\Delta_Y} \end{aligned} \tag{4.59}$$

The corresponding boundary conditions in $u_{n,i,j}$ ($n \in [0, N_t]$, $i \in [0, N_S]$, $j \in [0, N_Y]$) are

- Boundary Condition for t=0:

$$u[0,i,:] = \max(S_i - K, 0), \quad \forall i \in [0, N_S] \tag{4.60}$$

- Boundary Condition for $S = S_{min} = 0$:

$$u[:,0,:] = f(0) = 0, \quad \forall n \in [0, N_t], \quad \forall j \in [0, N_Y] \tag{4.61}$$

- Boundary Condition for $S = S_{max}$:

  We have $\frac{\partial^2 u}{\partial S^2} = 0$. We apply it to the equation (4.53)

$$\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S}+\alpha(m-Y)\frac{\partial u}{\partial Y}+\frac{1}{2}k^2Y\frac{\partial^2 u}{\partial Y^2}-ru+\rho g_2(s,y,t)Sk\sqrt{Y}\frac{\partial^2 u}{\partial S\partial Y} \tag{4.62}$$

  Here we use central differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,-1,j}}{\partial Y} \approx \frac{u_{n,-1,j+1}-u_{n,-1,j-1}}{2\Delta_Y} \tag{4.63}$$

$$\frac{\partial^2 u_{n,-1,j}}{\partial Y^2} \approx \frac{u_{n,-1,j+1} - 2u_{n,-1,j} + u_{n,-1,j-1}}{\Delta_Y^2} \tag{4.64}$$

We use forward differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,-1,j}}{\partial S} \approx \frac{u_{n,-1,j} - u_{n,-2,j}}{\Delta_S} \tag{4.65}$$

For $\frac{\partial^2 u}{\partial S \partial Y}$ we combine central differences and forward differences.

$$\frac{\partial^2 u_{n,-1,j}}{\partial S \partial Y} \approx \frac{\partial}{\partial S} \frac{u_{n,-1,j+1} - u_{n,-1,j-1}}{2\Delta_Y} \tag{4.66}$$

$$= \frac{1}{2\Delta_Y} \left(\frac{\partial u_{n,-1,j+1}}{\partial S} - \frac{\partial u_{n,-1,j-1}}{\partial S} \right) \tag{4.67}$$

$$\approx \frac{1}{2\Delta_Y} \left(\frac{u_{n,-1,j+1} - u_{n,-2,j+1}}{\Delta_S} - \frac{u_{n,-1,j-1} - u_{n,-2,j-1}}{\Delta_S} \right) \tag{4.68}$$

$$= \frac{u_{n,-1,j+1} - u_{n,-2,j+1} - u_{n,-1,j-1} + u_{n,-2,j-1}}{2\Delta_S \Delta_Y} \tag{4.69}$$

- Boundary Condition for $Y = Y_{max}$:

  We have $\frac{\partial^2 u}{\partial Y^2} = 0$. We apply it to the equation (4.53)

$$\frac{\partial u}{\partial t} = g_1(s,y,t) S \frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2 \frac{\partial^2 u}{\partial S^2} + \alpha(m - Y)\frac{\partial u}{\partial Y} - ru + \rho g_2(s,y,t) S k \sqrt{Y} \frac{\partial^2 u}{\partial S \partial Y} \tag{4.70}$$

  Here we use central differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,i,-1}}{\partial S} \approx \frac{u_{n,i+1,-1} - u_{n,i-1,-1}}{2\Delta_S} \tag{4.71}$$

$$\frac{\partial^2 u_{n,i,-1}}{\partial S^2} \approx \frac{u_{n,i+1,-1} - 2u_{n,i,-1} + u_{n,i-1,-1}}{\Delta_S^2} \tag{4.72}$$

We use forward differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,i,-1}}{\partial Y} \approx \frac{u_{n,i,-1} - u_{n,i,-2}}{\Delta_Y} \tag{4.73}$$

For $\frac{\partial^2 u}{\partial S \partial Y}$ we combine central differences and forward differences.

$$\begin{aligned}
\frac{\partial^2 u_{n,i,-1}}{\partial S \partial Y} &\approx \frac{\partial}{\partial S} \frac{u_{n,i,-1} - u_{n,i,-2}}{\Delta_Y} && (4.74)\\
&= \frac{1}{\Delta_Y} (\frac{\partial u_{n,i,-1}}{\partial S} - \frac{\partial u_{n,i,-2}}{\partial S}) && (4.75)\\
&\approx \frac{1}{\Delta_Y} (\frac{u_{n,i+1,-1} - u_{n,i-1,-1}}{2\Delta_S} - \frac{u_{n,i+1,-2} - u_{n,i-1,-2}}{2\Delta_S}) && (4.76)\\
&= \frac{u_{n,i+1,-1} - u_{n,i-1,-1} - u_{n,i+1,-2} + u_{n,i-1,-2}}{2\Delta_S \Delta_Y} && (4.77)
\end{aligned}$$

- Boundary Condition for $Y = Y_{min} = 0$:

  The PDE reduces to

$$\frac{\partial u}{\partial t} = -ru + g_1(s,y,t) S \frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2 \frac{\partial^2 u}{\partial S^2} + \alpha m \frac{\partial u}{\partial Y} \tag{4.78}$$

  Here we use central differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,i,0}}{\partial S} \approx \frac{u_{n,i+1,0} - u_{n,i-1,0}}{2\Delta_S} \tag{4.79}$$

$$\frac{\partial^2 u_{n,i,0}}{\partial S^2} \approx \frac{u_{n,i+1,0} - 2u_{n,i,0} + u_{n,i-1,0}}{\Delta_S^2} \tag{4.80}$$

  We use forward differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,i,0}}{\partial Y} \approx \frac{u_{n,i,1} - u_{n,i,0}}{\Delta_Y} \tag{4.81}$$

The following is the corresponding algorithm.

Algorithm 8 Optimization algorithm for 2-dimensional neural network PDE with Heston

procedure
  $S \leftarrow ones(N_S + 1)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i\Delta_S$
  end for
  for $j \leftarrow 1$ `to` N_Y **do**
    $Y[j] = j\Delta_Y$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow zeros(N + 1, N_S + 1, N_Y + 1)$
    $u[0, :, :] \leftarrow max(S - K, 0)$
    for $t \leftarrow 1$ `to` N **do**
      $u[t, 0, : -1] \leftarrow 0$
      $g_1, g_2 \leftarrow g(S, K, t)$
      $u_y \leftarrow (u[t, -1, 2 :] - u[t, -1, : -2])/(2\Delta_Y)$
      $u_s \leftarrow (u[t, -1, 1 : -1] - u[t, -2, 1 : -1])/(\Delta_S)$
      $u_{yy} \leftarrow (u[t, -1, 2 :] - 2u[t, -1, 1 : -1] + u[t, -1, : -2])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, -1, 2 :] - u[t, -2, 2 :] - u[t, -1, : -2] + u[t, -1, : -2])/(2\Delta_S\Delta_Y)$
      $L_1 \leftarrow -ru[t, -1, 1 : -1] + g_1 S[-1]u_s + \alpha(m - Y[1 : -1])u_y + \frac{1}{2}k^2 Y[1 : -1]u_{yy} + \rho g_2 S[-1]k\sqrt{Y[1 : -1]}u_{sy}$
      $u[t, -1, : -1] \leftarrow L_1\Delta_t + u[t, 1 : -1, -1]$
      $u_y \leftarrow (u[t, 1 : -1, -1] - u[t, 1 : -1, -2])/(\Delta_Y)$
      $u_s \leftarrow (u[t, 2 :, -1] - u[t, : -2, -1])/(2\Delta_S)$
      $u_{ss} \leftarrow (u[t, 2 :, -1] - 2u[t, 1 : -1, -1] + u[t, : -2, -1])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, 2 :, -1] - u[t, : -2, -1] - u[t, 2 :, -2] + u[t, : -2, -2])/(2\Delta_S\Delta_Y)$
      $L_2 \leftarrow -ru[t, 1 : -1, -1] + g_1 S[1 : -1]u_s + \frac{1}{2}(g_2 S[1 : -1])^2 u_{ss} + \alpha(m - Y[-1])u_y + \rho g_2 S[1 : -1]k\sqrt{Y[-1]}u_{sy}$
      $u[t, 1 : -1, -1] \leftarrow L_2\Delta_t + u[t, 1 : -1, -1]$
      $u_y \leftarrow (u[t, 1 : -1, 1] - u[t, 1 : -1, 0])/(\Delta_Y)$
      $u_s \leftarrow (u[t, 2 :, 0] - u[t, : -2, 0])/(2\Delta_S)$
      $u_{ss} \leftarrow (u[t, 2 :, 0] - 2u[t, 1 : -1, 0] + u[t, : -2, 0])/(\Delta_S^2)$
      $L_3 \leftarrow -ru[t, 1 : -1, 0] + g_1 S[1 : -1]u_s + \frac{1}{2}(g_2 S[1 : -1])^2 u_{ss} + \alpha m u_y$
      $u[t, 1 : -1, 0] \leftarrow L_3\Delta_t + u[t, 1 : -1, 0]$
      $u_y \leftarrow (u[t, 1 : -1, 2 :] - u[t, 1 : -1, : -2])/(2\Delta_Y)$
      $u_s \leftarrow (u[t, 2 :, 1 : -1] - u[t, : -2, 1 : -1])/(2\Delta_S)$
      $u_{yy} \leftarrow (u[t, 1 : -1, 2 :] - 2u[t, 1 : -1, 1 : -1] + u[t, 1 : -1, : -2])/(\Delta_Y^2)$
      $u_{ss} \leftarrow (u[t, 2 :, 1 : -1] - 2u[t, 1 : -1, 1 : -1] + u[t, : -2, 1 : -1])/(\Delta_S^2)$

$u_{sy} \leftarrow (u[t, 2\ :, 2\ :] - u[t, :\ -2, 2\ :] - u[t, 2\ :, :\ -2] + u[t, :\ -2, :\ -2])/(4\Delta_S\Delta_Y)$

$L_4 \leftarrow \frac{1}{2}(g_2 S[1\ :\ -1])^2 u_{ss} + \rho g_2 S[1\ :\ -1] k\sqrt{Y[1:-1]} u_{sy} + \frac{1}{2}k^2 Y[1\ :\ -1] u_{yy} - ru[t, 1:-1, 1:-1] + g_1 S[1:-1] u_s + \alpha(m - Y[1:-1]) u_y$

$u[t, 1:-1, 1:-1] \leftarrow L_4\Delta_t + u[t, 1:-1, 1:-1]$

$h_0 \leftarrow max(S - K, 0)$

$u \leftarrow max(u, h_0)$

end for

$P \leftarrow interpolate(u[-1, I_0, :], Y0)$

$loss \leftarrow (P - P_{market})^2$

loss.backward()

optimizer.step()

end for

end procedure

4.4.3 2-Dimensional Neural-network PDE model

Additionally, we would like to give more freedom for Y_t in the PDE. So we use a neural network $g(S_t, Y_t, t)$ with an output dimension of 4 (g_1, g_2, g_3, and g_4) to replace the drift and diffusion of the price SDE in the Heston model. We call this model 2D-NN.

The equations for the 2-dimensional SDEs are

$$dS_t = g_1(t, S_t, Y_t)S_t dt + g_2(t, S_t, Y_t)S_t dW_t^s \tag{4.82}$$

$$dY_t = g_3(t, S_t, Y_t)dt + g_4(t, S_t, Y_t)dW_t^y \tag{4.83}$$

$$dW_t^S dW_t^Y = \rho dt \tag{4.84}$$

Note here Y_t can have both positive and negative values.

Consider the option price with a discounted payoff

$$u(t,s,y) = \mathbb{E}\left[e^{-r(T-t)} f(S_T, Y_T) | S_t = s, Y_t = y\right] \tag{4.85}$$

With tower property, we have

$$\begin{aligned} u(t,S_t,Y_t) &= \mathbb{E}\left[e^{-r(T-t)} f(S_T,Y_T)|S_t = S_t, Y_t = Y_t\right] \\ &= \mathbb{E}\left[e^{-rdt}\mathbb{E}\left(e^{-r(T-t-dt)} f(S_T,Y_T)|S_{t+dt} = S_{t+dt}, Y_{t+dt} = Y_{t+dt}\right)|S_t = S_t, Y_t = Y_t\right] \\ &= \mathbb{E}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})|S_t = S_t, Y_t = Y_t\right] \\ &= \mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})\right] \end{aligned} \tag{4.86}$$

Here using the Taylor series, we have

$$\begin{aligned} \mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})\right] &= \mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_t+dS, Y_t+dY)\right] \\ &\approx \mathbb{E}_{S_t,Y_t}[u(t,S_t,Y_t) + (e^{-rdt}(-r)u(t,S_t,Y_t) + e^{-rdt}\partial_t u(t,S_t,Y_t))dt \\ &+ e^{-rdt}(\partial_S u(t,S_t,Y_t)dS + \partial_Y u(t,S_t,Y_t)dY + \frac{1}{2}\partial_{SS} u(t,S_t,Y_t)[dS]^2 \\ &+ \frac{1}{2}\partial_{YY} u(t,S_t,Y_t)[dY]^2 + \partial_{SY} u(t,S_t,Y_t)[dSdY])] \end{aligned} \tag{4.87}$$

Here we have $\mathbb{E}(dS) = g_1 S dt$, $\mathbb{E}(dY) = g_3 dt$, $\mathbb{E}([dS]^2) = g_2^2 S^2 dt$, $\mathbb{E}([dY]^2) = g_4^2 dt$ and $\mathbb{E}([dSdY]) = \rho g_2 g_4 S dt$. So we have

$$\begin{aligned} \mathbb{E}_{S_t,Y_t}\left[e^{-rdt} u(t+dt, S_{t+dt}, Y_{t+dt})\right] \approx u(t,S_t,Y_t) + e^{-rdt}(-ru + \partial_t u + g_1 S\partial_S u + g_3\partial_Y u \\ + \frac{1}{2}g_2^2 S^2 \partial_{SS} u + \frac{1}{2}g_4^2 \partial_{YY} u + \rho g_2 g_4 S \partial_{SY} u)dt \end{aligned} \tag{4.88}$$

Compare this with equation (4.86), we have

$$\begin{aligned} u(t,S_t,Y_t) \approx u(t,S_t,Y_t) + e^{-rdt}(-ru + \partial_t u + g_1 S\partial_S u + g_3\partial_Y u \\ + \frac{1}{2}g_2^2 S^2 \partial_{SS} u + \frac{1}{2}g_4^2 \partial_{YY} u + \rho g_2 g_4 S \partial_{SY} u)dt \end{aligned} \tag{4.89}$$

Thus, we have

$$\begin{aligned} -\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2\frac{\partial^2 u}{\partial S^2} + g_3(s,y,t)\frac{\partial u}{\partial Y} \\ + \frac{1}{2}(g_4(s,y,t))^2\frac{\partial^2 u}{\partial Y^2} - ru + \rho g_2(s,y,t)g_4(s,y,t)S\frac{\partial^2 u}{\partial S \partial Y} \end{aligned} \tag{4.90}$$

with $u(0,s,y) = \mathbb{E}\left[e^{-rT}f(S_T,Y_T)|S_0 = s, Y_0 = y\right]$ and $u(T,s,y) = f(s,y)$.

In addition, we use the coordinate transformation $t \to (T-t)$, then the PDE becomes

$$\begin{aligned} \frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2\frac{\partial^2 u}{\partial S^2} + g_3(s,y,t)\frac{\partial u}{\partial Y} \\ + \frac{1}{2}(g_4(s,y,t))^2\frac{\partial^2 u}{\partial Y^2} - ru + \rho g_2(s,y,t)g_4(s,y,t)S\frac{\partial^2 u}{\partial S \partial Y} \end{aligned} \tag{4.91}$$

with $u(T,s,y) = \mathbb{E}\left[e^{-rT}f(S_T,Y_T)|S_0 = s, Y_0 = y\right]$ and $u(0,s,y) = f(s,y)$.

Using the finite difference method, we have

$$\frac{\partial u_{n,i,j}}{\partial S} = \frac{u_{n,i+1,j} - u_{n,i-1,j}}{2\Delta_S} \tag{4.92}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial S^2} = \frac{u_{n,i+1,j} - 2u_{n,i,j} + u_{n,i-1,j}}{\Delta_S^2} \tag{4.93}$$

$$\frac{\partial u_{n,i,j}}{\partial Y} = \frac{u_{n,i,j+1} - u_{n,i,j-1}}{2\Delta_Y} \tag{4.94}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial Y^2} = \frac{u_{n,i,j+1} - 2u_{n,i,j} + u_{n,i,j-1}}{\Delta_Y^2} \tag{4.95}$$

$$\frac{\partial^2 u_{n,i,j}}{\partial S \partial Y} = \frac{u_{n,i+1,j+1} + u_{n,i-1,j-1} - u_{n,i-1,j+1} - u_{n,i+1,j-1}}{4\Delta_S \Delta_Y} \tag{4.96}$$

So the final update equation is

$$\begin{aligned} u_{n+1,i,j} = u_{n,i,j} + \Delta_t (g_1(s,y,t) S_i \frac{u_{n,i+1,j} - u_{n,i-1,j}}{2\Delta_S} \\ + \frac{1}{2}(g_2(s,y,t)S_i)^2 \frac{u_{n,i+1,j} - 2u_{n,i,j} + u_{n,i-1,j}}{\Delta_S^2} + g_3(s,y,t)\frac{u_{n,i,j+1} - u_{n,i,j-1}}{2\Delta_Y} \\ + \frac{1}{2}(g_4(s,y,t))^2 \frac{u_{n,i,j+1} - 2u_{n,i,j} + u_{n,i,j-1}}{\Delta_Y^2} - r u_{n,i,j} \\ + \rho g_2(s,y,t) g_4(s,y,t) S_i \frac{u_{n,i+1,j+1} + u_{n,i-1,j-1} - u_{n,i-1,j+1} - u_{n,i+1,j-1}}{4\Delta_S \Delta_Y} \end{aligned} \tag{4.97}$$

The corresponding boundary conditions in $u_{n,i,j}$ ($n \in [0, N_t]$, $i \in [0, N_S]$, $j \in [-N_Y, N_Y]$) are

- Boundary Condition for t=0:

$$u[0, i, :] = \max(S_i - K, 0), \quad \forall i \in [0, N_S] \tag{4.98}$$

- Boundary Condition for $S = S_{min} = 0$:

$$u[:, 0, :] = f(0) = 0, \quad \forall n \in [0, N_t], \quad \forall j \in [-N_Y, N_Y] \tag{4.99}$$

- Boundary Condition for $S = S_{max}$:

We have $\frac{\partial^2 u}{\partial S^2} = 0$. We apply it to the equation (4.91)

$$\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + g_3(s,y,t)\frac{\partial u}{\partial Y} + \frac{1}{2}(g_4(s,y,t))^2\frac{\partial^2 u}{\partial Y^2} - ru \\ +\rho g_2(s,y,t)g_4(s,y,t)S\frac{\partial^2 u}{\partial S \partial Y} \tag{4.100}$$

Here we use central differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,-1,j}}{\partial Y} \approx \frac{u_{n,-1,j+1} - u_{n,-1,j-1}}{2\Delta_Y} \tag{4.101}$$

$$\frac{\partial^2 u_{n,-1,j}}{\partial Y^2} \approx \frac{u_{n,-1,j+1} - 2u_{n,-1,j} + u_{n,-1,j-1}}{\Delta_Y^2} \tag{4.102}$$

We use forward differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,-1,j}}{\partial S} \approx \frac{u_{n,-1,j} - u_{n,-2,j}}{\Delta_S} \tag{4.103}$$

For $\frac{\partial^2 u}{\partial S \partial Y}$ we combine central differences and forward differences.

$$\frac{\partial^2 u_{n,-1,j}}{\partial S \partial Y} \approx \frac{\partial}{\partial S}\frac{u_{n,-1,j+1} - u_{n,-1,j-1}}{2\Delta_Y} \tag{4.104}$$

$$= \frac{1}{2\Delta_Y}\left(\frac{\partial u_{n,-1,j+1}}{\partial S} - \frac{\partial u_{n,-1,j-1}}{\partial S}\right) \tag{4.105}$$

$$\approx \frac{1}{2\Delta_Y}\left(\frac{u_{n,-1,j+1} - u_{n,-2,j+1}}{\Delta_S} - \frac{u_{n,-1,j-1} - u_{n,-2,j-1}}{\Delta_S}\right) \tag{4.106}$$

$$= \frac{u_{n,-1,j+1} - u_{n,-2,j+1} - u_{n,-1,j-1} + u_{n,-2,j-1}}{2\Delta_S\Delta_Y} \tag{4.107}$$

- Boundary Condition for $Y = Y_{max}$:

We have $\frac{\partial^2 u}{\partial Y^2} = 0$. We apply it to the equation (4.91)

$$\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2\frac{\partial^2 u}{\partial S^2} + g_3(s,y,t)\frac{\partial u}{\partial Y} - ru \\ +\rho g_2(s,y,t)g_4(s,y,t)S\frac{\partial^2 u}{\partial S \partial Y} \tag{4.108}$$

Here we use central differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,i,-1}}{\partial S} \approx \frac{u_{n,i+1,-1} - u_{n,i-1,-1}}{2\Delta_S} \tag{4.109}$$

$$\frac{\partial^2 u_{n,i,-1}}{\partial S^2} \approx \frac{u_{n,i+1,-1} - 2u_{n,i,-1} + u_{n,i-1,-1}}{\Delta_S^2} \tag{4.110}$$

We use forward differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,i,-1}}{\partial Y} \approx \frac{u_{n,i,-1} - u_{n,i,-2}}{\Delta_Y} \tag{4.111}$$

For $\frac{\partial^2 u}{\partial S \partial Y}$ we combine central differences and forward differences.

$$\frac{\partial^2 u_{n,i,-1}}{\partial S \partial Y} \approx \frac{\partial}{\partial S}\frac{u_{n,i,-1} - u_{n,i,-2}}{\Delta_Y} \tag{4.112}$$

$$= \frac{1}{\Delta_Y}(\frac{\partial u_{n,i,-1}}{\partial S} - \frac{\partial u_{n,i,-2}}{\partial S}) \tag{4.113}$$

$$\approx \frac{1}{\Delta_Y}(\frac{u_{n,i+1,-1} - u_{n,i-1,-1}}{2\Delta_S} - \frac{u_{n,i+1,-2} - u_{n,i-1,-2}}{2\Delta_S}) \tag{4.114}$$

$$= \frac{u_{n,i+1,-1} - u_{n,i-1,-1} - u_{n,i+1,-2} + u_{n,i-1,-2}}{2\Delta_S\Delta_Y} \tag{4.115}$$

- Boundary Condition for $Y = Y_{min}$:

We have $\frac{\partial^2 u}{\partial Y^2} = 0$ as well. We apply it to the equation (4.91)

$$\frac{\partial u}{\partial t} = g_1(s,y,t)S\frac{\partial u}{\partial S} + \frac{1}{2}(g_2(s,y,t)S)^2\frac{\partial^2 u}{\partial S^2} + g_3(s,y,t)\frac{\partial u}{\partial Y} - ru \\ +\rho g_2(s,y,t)g_4(s,y,t)S\frac{\partial^2 u}{\partial S \partial Y} \tag{4.116}$$

Here we use central differences for $\frac{\partial u}{\partial S}$

$$\frac{\partial u_{n,i,0}}{\partial S} \approx \frac{u_{n,i+1,0} - u_{n,i-1,0}}{2\Delta_S} \tag{4.117}$$

$$\frac{\partial^2 u_{n,i,0}}{\partial S^2} \approx \frac{u_{n,i+1,0} - 2u_{n,i,0} + u_{n,i-1,0}}{\Delta_S^2} \tag{4.118}$$

We use forward differences for $\frac{\partial u}{\partial Y}$

$$\frac{\partial u_{n,i,0}}{\partial Y} \approx \frac{u_{n,i,1} - u_{n,i,0}}{\Delta_Y} \tag{4.119}$$

For $\frac{\partial^2 u}{\partial S \partial Y}$ we combine central differences and forward differences.

$$\frac{\partial^2 u_{n,i,0}}{\partial S \partial Y} \approx \frac{\partial}{\partial S}\frac{u_{n,i,1} - u_{n,i,0}}{\Delta_Y} \tag{4.120}$$

$$= \frac{1}{\Delta_Y}(\frac{\partial u_{n,i,1}}{\partial S} - \frac{\partial u_{n,i,0}}{\partial S}) \tag{4.121}$$

$$\approx \frac{1}{\Delta_Y}(\frac{u_{n,i+1,1} - u_{n,i-1,1}}{2\Delta_S} - \frac{u_{n,i+1,0} - u_{n,i-1,0}}{2\Delta_S}) \tag{4.122}$$

$$= \frac{u_{n,i+1,1} - u_{n,i-1,1} - u_{n,i+1,0} + u_{n,i-1,0}}{2\Delta_S\Delta_Y} \tag{4.123}$$

The following is the corresponding algorithm.

Algorithm 9 Optimization algorithm for 2-dimensional neural network PDE

procedure
  $S \leftarrow ones(N_S + 1)$
  for $i \leftarrow 1$ `to` N_S **do**
    $S[i] = i\Delta_S$
  end for
  for $j \leftarrow 1$ `to` N_Y **do**
    $Y[j] = j\Delta_Y$
  end for
  for $epoch \leftarrow 1$ `to` $epochs$ **do**
    $u \leftarrow zeros(N + 1, N_S + 1, N_Y + 1)$
    $u[0, :, :] \leftarrow max(S - K, 0)$
    for $t \leftarrow 1$ `to` N **do**
      $u[t, 0, : -1] \leftarrow 0$
      $g_1, g_2, g_3, g_4 \leftarrow g(S, K, t)$
      $u_y \leftarrow (u[t, -1, 2 :] - u[t, -1, : -2])/(2\Delta_Y)$
      $u_s \leftarrow (u[t, -1, 1 : -1] - u[t, -2, 1 : -1])/(\Delta_S)$
      $u_{yy} \leftarrow (u[t, -1, 2 :] - 2u[t, -1, 1 : -1] + u[t, -1, : -2])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, -1, 2 :] - u[t, -2, 2 :] - u[t, -1, : -2] + u[t, -1, : -2])/(2\Delta_S\Delta_Y)$
      $L_1 \leftarrow -ru[t, -1, 1 : -1] + g_1 S[-1]u_s + g_3 u_y + \frac{1}{2}(g_4)^2 u_{yy} + \rho g_2 g_4 S[-1]u_{sy}$
      $u[t, -1, : -1] \leftarrow L_1\Delta_t + u[t, 1 : -1, -1]$
      $u_y \leftarrow (u[t, 1 : -1, -1] - u[t, 1 : -1, -2])/(\Delta_Y)$
      $u_s \leftarrow (u[t, 2 :, -1] - u[t, : -2, -1])/(2\Delta_S)$
      $u_{ss} \leftarrow (u[t, 2 :, -1] - 2u[t, 1 : -1, -1] + u[t, : -2, -1])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, 2 :, -1] - u[t, : -2, -1] - u[t, 2 :, -2] + u[t, : -2, -2])/(2\Delta_S\Delta_Y)$
      $L_2 \leftarrow -ru[t, 1 : -1, -1] + g_1 S[1 : -1]u_s + \frac{1}{2}(g_2 S[1 : -1])^2 u_{ss} + g_3 u_y + \rho g_2 g_4 S[1 : -1]u_{sy}$
      $u[t, 1 : -1, -1] \leftarrow L_2\Delta_t + u[t, 1 : -1, -1]$
      $u_y \leftarrow (u[t, 1 : -1, 1] - u[t, 1 : -1, 0])/(\Delta_Y)$
      $u_s \leftarrow (u[t, 2 :, 0] - u[t, : -2, 0])/(2\Delta_S)$
      $u_{ss} \leftarrow (u[t, 2 :, 0] - 2u[t, 1 : -1, 0] + u[t, : -2, 0])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, 2 :, 1] - u[t, : -2, 1] - u[t, 2 :, 0] + u[t, : -2, 0])/(2\Delta_S\Delta_Y)$
      $L_3 \leftarrow -ru[t, 1 : -1, 0] + g_1 S[1 : -1]u_s + \frac{1}{2}(g_2 S[1 : -1])^2 u_{ss} + g_3 u_y + \rho g_2 g_4 S[1 : -1]u_{sy}$
      $u[t, 1 : -1, 0] \leftarrow L_3\Delta_t + u[t, 1 : -1, 0]$
      $u_y \leftarrow (u[t, 1 : -1, 2 :] - u[t, 1 : -1, : -2])/(2\Delta_Y)$
      $u_s \leftarrow (u[t, 2 :, 1 : -1] - u[t, : -2, 1 : -1])/(2\Delta_S)$
      $u_{yy} \leftarrow (u[t, 1 : -1, 2 :] - 2u[t, 1 : -1, 1 : -1] + u[t, 1 : -1, : -2])/(\Delta_Y^2)$
      $u_{ss} \leftarrow (u[t, 2 :, 1 : -1] - 2u[t, 1 : -1, 1 : -1] + u[t, : -2, 1 : -1])/(\Delta_S^2)$
      $u_{sy} \leftarrow (u[t, 2 :, 2 :] - u[t, : -2, 2 :] - u[t, 2 :, : -2] + u[t, : -2, : -2])/(4\Delta_S\Delta_Y)$
      $L_4 \leftarrow \frac{1}{2}(g_2 S[1 : -1])^2 u_{ss} + \rho g_2 g_4 S[1 : -1]u_{sy} + \frac{1}{2}(g_4)^2 u_{yy} - ru[t, 1 : -1, 1 : -1] + g_1 S[1 : -1]u_s + g_3 u_y$

$u[t, 1:-1, 1:-1] \leftarrow L_4 \Delta_t + u[t, 1:-1, 1:-1]$
$h_0 \leftarrow max(S - K, 0)$
$u \leftarrow max(u, h_0)$
end for
$P \leftarrow interpolate(u[-1, I_0, :], Y0)$
$loss \leftarrow (P - P_{market})^2$
loss.backward()
optimizer.step()
end for
end procedure

4.5 Experiments and Results

We test the performance of our models on various datasets.

4.5.1 American Stock Options

In this experiment, we randomly select 100 stocks from S&P500.

ABC, ABT, AES, ALB, AMGN, AMT, AMZN, ANSS, APD, AVGO, AWK, BA, BBY, BDX, BIIB, BK, BKNG, BKR, BLK, CFG, CHTR, CMCSA, CMG, COF, COST, CPRT, DE, DIS, DISCK, DOV, DTE, EFX, EMR, ENPH, EOG, EQR, EXC, EXPD, EXR, FCX, FE, FIS, FRC, GOOGL, GPC, GPS, GS, HOLX, JBHT, KMB, KMI, KMX, LH, LIN, LNC, LRCX, LUV, MKC, MNST, NI, NRG, NTRS, NVDA, NWSA, OKE, OMC, PAYX, PCAR, PEAK, PENN, PEP, PG, PGR, PKI, PNC, PNW, PPG, PWR, PYPL, REG, RMD, ROK, ROST, SBUX, SHW, SLB, STX, SYF, TFX, TGT, TSCO, TSLA, TXT, TYL, UA, VIAC, VMC, WRK, WYNN, XLNX

For these 100 American options, we solved 3 special cases below:

- Some stocks have split history. The close price adjusted for the split but the historical options prices didn't. So we need to use the close price multiplies

the split factor to recover the true historical stock price before calculating the volatility.

- Some stocks changed their ticker name from 09/01/2017 to now. Below is a list of tickers in this situation.
  - PCLN → BKNG
  - BHGE → BKR
  - PX → LIN
  - HCP → HCP
  - CBS → VIAC
- Some options have 2 or 3 similar tickers and they have different current stock prices assumed. We only use the first ticker in the list below.
  - 'ABT', 'ABT1'
  - 'APD', 'APD1'
  - 'AVGO', 'AVGO1'
  - 'BHGE', 'BHGE1'
  - 'BIIB', 'BIIB1'
  - 'CHTR', 'CHTR1', 'CHTR2'
  - 'OKE', 'OKE1'
  - 'REG', 'REG1'
  - 'SLB', 'SLB1'
  - 'TSLA', 'TSLA1'

We start from 09/01/2017 and do the following experiments.

The notations for the models we will compare are

- **BS-Scalar:** The Black-Scholes PDE model where we estimate a scalar implied volatility from all training data. It is a single parameter model for all (maturity, strike) pairs.
- **BS-LV:** The Black-Scholes PDE model where we estimate the implied volatility for each (maturity, strike) pair.
- **1D-NN:** The 1-dimensional neural-network PDE model in section 4.2.
- **Heston:** The Heston PDE model in section 4.4.1.
- **LV-NN:** The neural-network local volatility PDE model in section 4.3.
- **2D-NN-Heston:** The 2-dimensional neural-network PDE model where Y_t follows the SDE of the Heston model in section 4.4.2.
- **2D-NN:** The 2-dimensional neural-network PDE model in section 4.4.3.

Intraday Out-of-sample Prediction

On 2017-09-01, for each stock, we select 80% of pairs (Call and Put pairs) for training and the remaining 20% data for testing. Below are the intraday pricing results taking the average of the 100 stocks.

Experiment of intraday prediction, part 1.						
	BS Scalar		Heston		1D-NN	
	Call	Put	Call	Put	Call	Put
2017-09-01	0.433 1.369 40.063%	0.448 1.315 38.253%	0.401 1.166 33.623%	0.416 1.188 34.952%	0.427 1.353 40.875%	0.425 1.149 36.952%

Table 4.1: Experiment of intraday prediction for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of intraday prediction, part 2.						
2017-09-01	NNLV		2D-NN-Heston		2D-NN	
	Call	Put	Call	Put	Call	Put
2017-09-01	0.423 1.220 39.194%	0.424 1.127 37.005%	0.358 1.026 29.633%	0.340 1.084 30.148%	0.345 0.857 32.188%	0.352 0.976 28.121%

Table 4.2: Experiment of intraday prediction for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

We have some selected figures to visualize the performance of these models.

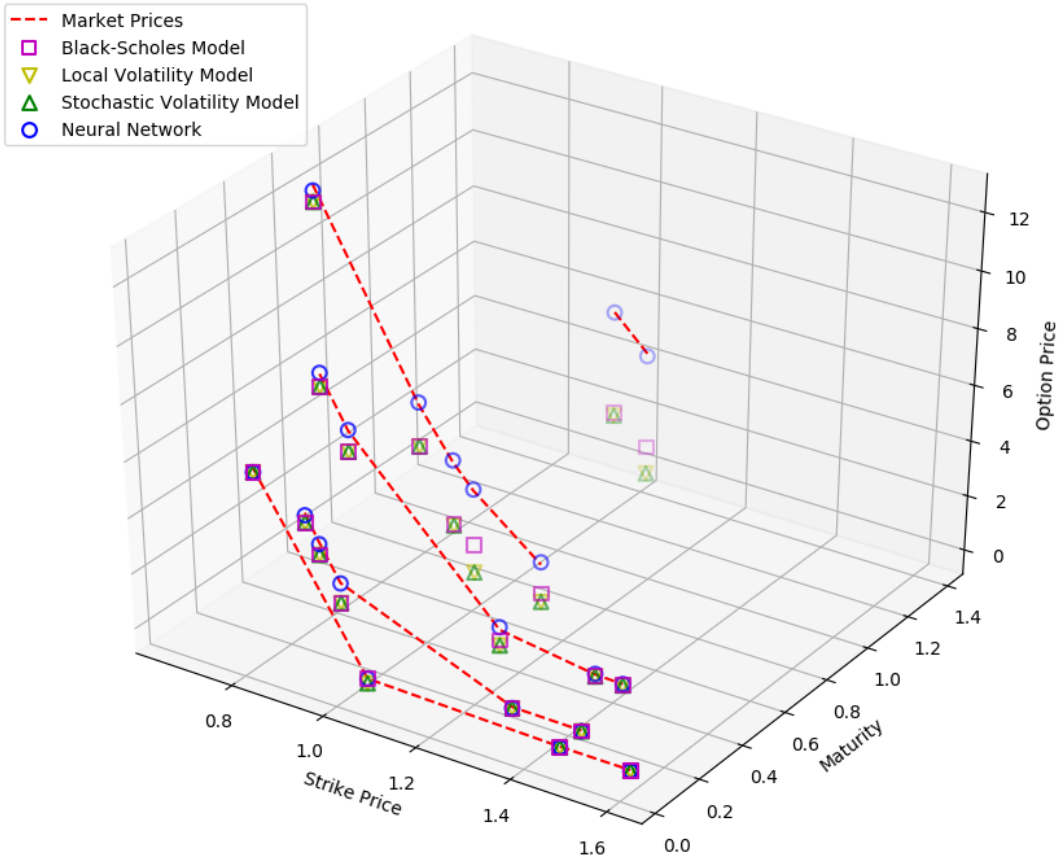


Figure 4.1: The intraday out of sample call option curve for ticker "BKR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

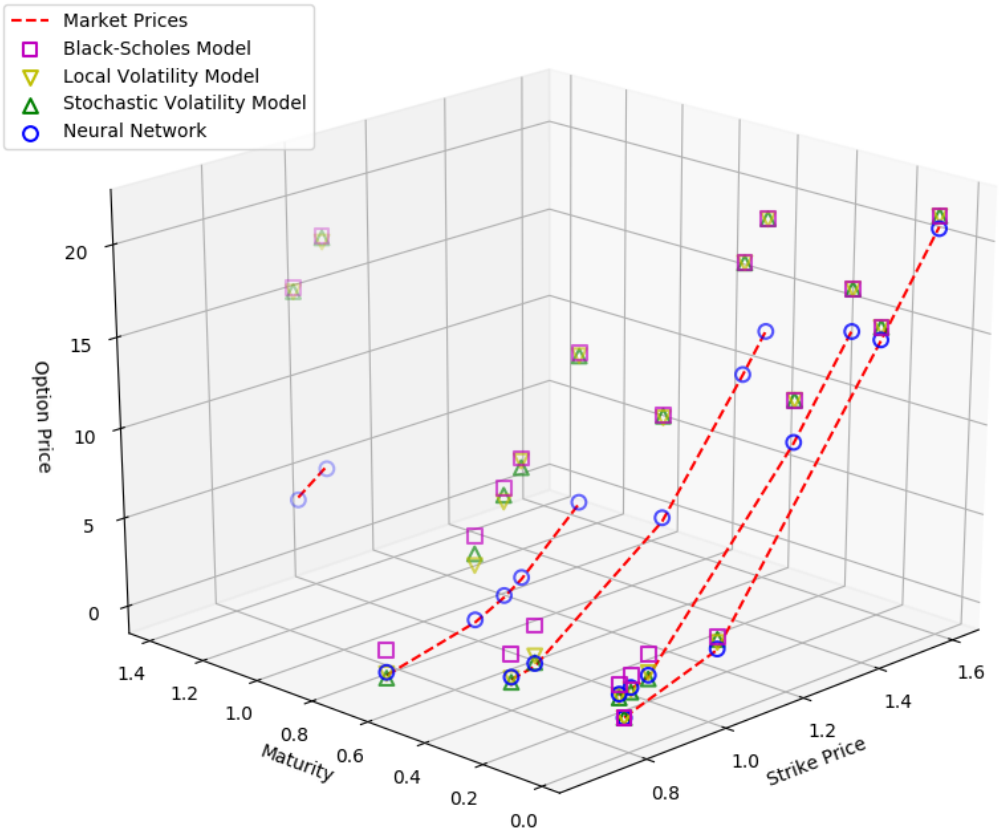


Figure 4.2: The intraday out of sample put option curve for ticker "BKR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

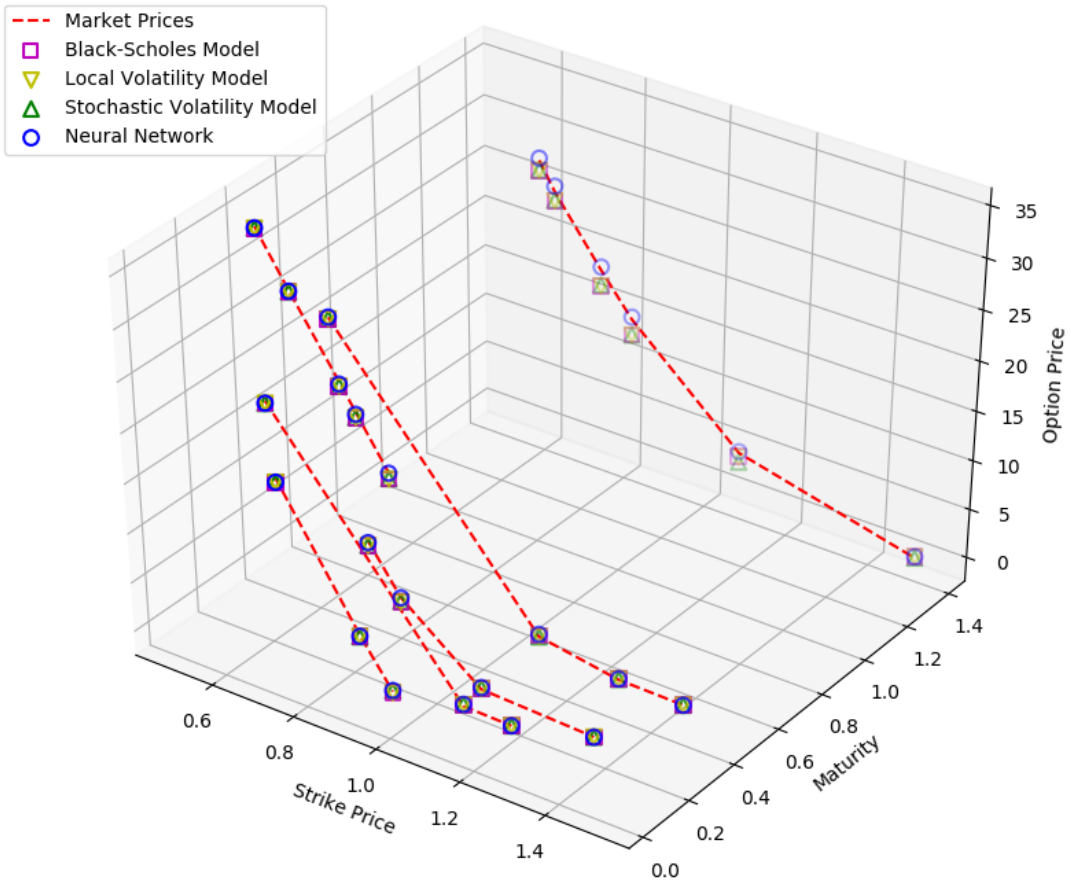


Figure 4.3: The intraday out of sample call option curve for ticker "PCAR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

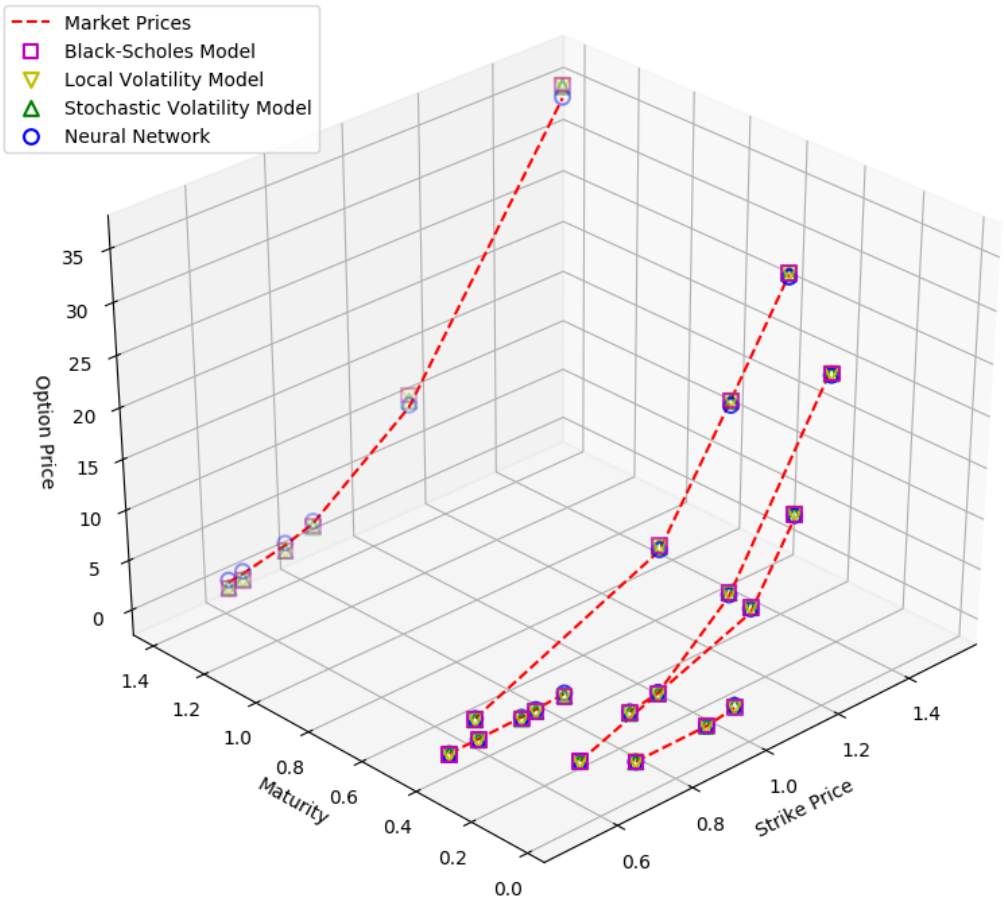


Figure 4.4: The intraday out of sample put option curve for ticker "PCAR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

We can observe from the figures that the 2D-NN model calibrates the market price more accurately than the other three models, especially when the maturity is large.

The Next-day Prediction

Next, we focus on the next-day pricing. We train the model on all the options data on 2017-09-01 and test the model performance on the next trading day (2017-09-05).

Below are the next-day pricing results taking the average of the 100 stocks.

Experiment of the next-day prediction, part 1.						
	BS Scalar		BS-LV		Heston	
	Call	Put	Call	Put	Call	Put
2017-09-05	0.400	0.433	0.307	0.314	0.372	0.391
	1.349	1.237	0.487	0.747	1.136	1.106
	38.651%	36.421%	36.757%	36.694%	31.416%	33.596%

Table 4.3: Experiment of the next-day prediction for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of the next-day prediction, part 2.				
	1D-NN		NNLV	
	Call	Put	Call	Put
2017-09-05	0.388	0.410	0.387	0.419
	1.201	1.110	1.136	1.156
	34.955%	34.985%	35.249%	35.586%

Table 4.4: Experiment of the next-day prediction for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of the next-day prediction, part 3.				
	2D-NN-Heston		2D-NN	
	Call	Put	Call	Put
2017-09-05	0.352 1.253 27.778%	0.327 0.704 29.756%	0.264 0.552 27.597%	0.286 0.446 25.475%

Table 4.5: Experiment of the next-day prediction for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Below are selected figures to visualize the performance of these models.

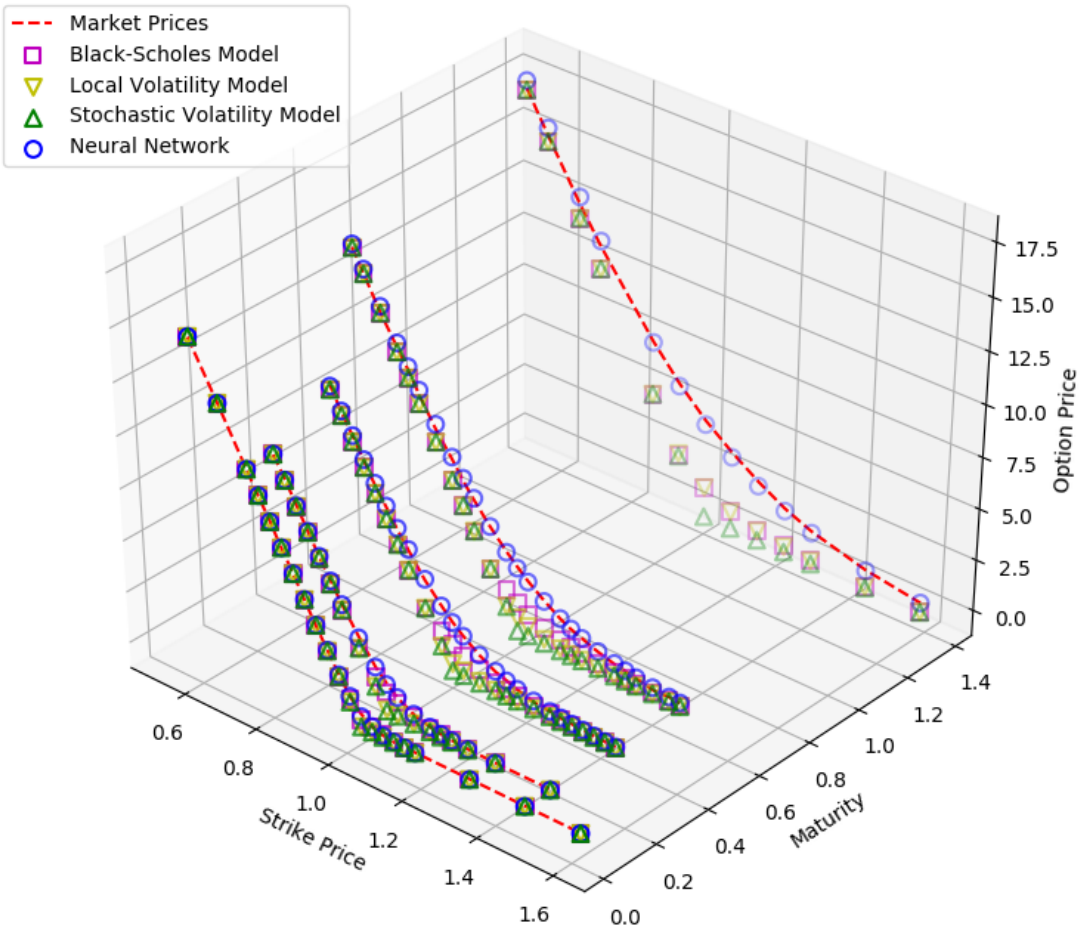


Figure 4.5: The next day out of sample call option curve for ticker "BKR". The date is 2017-09-05. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

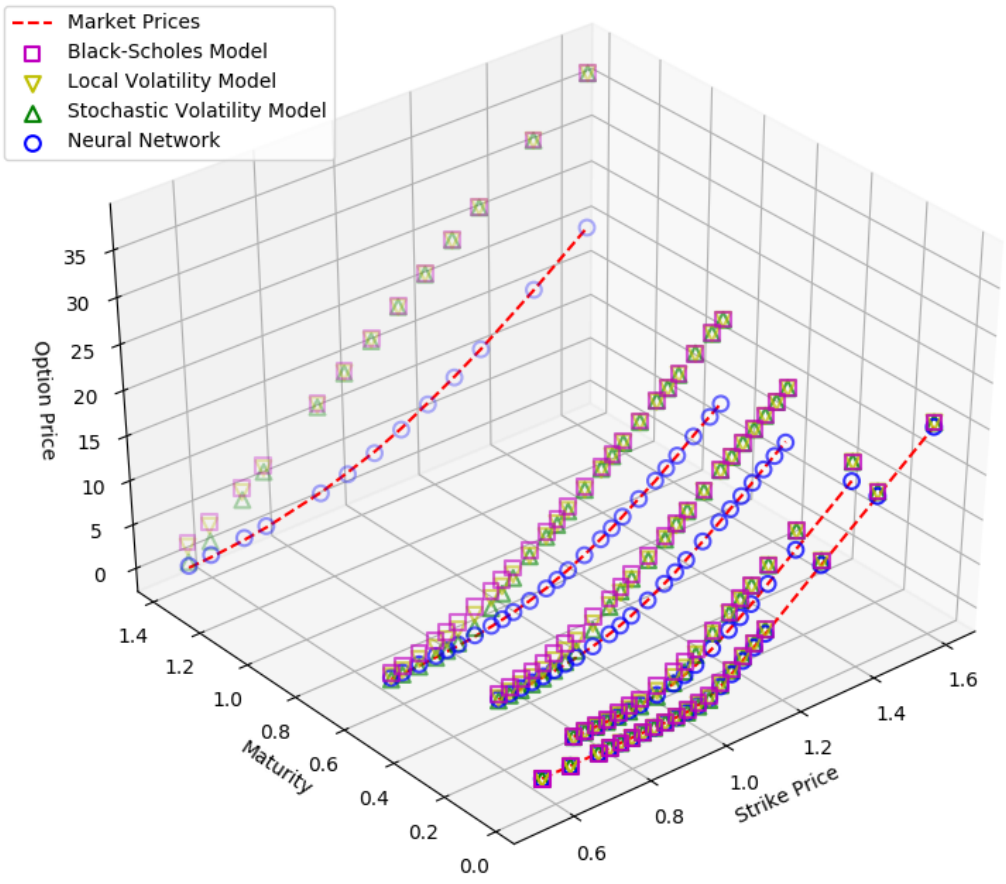


Figure 4.6: The next day out of sample put option curve for ticker "BKR". The date is 2017-09-05. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

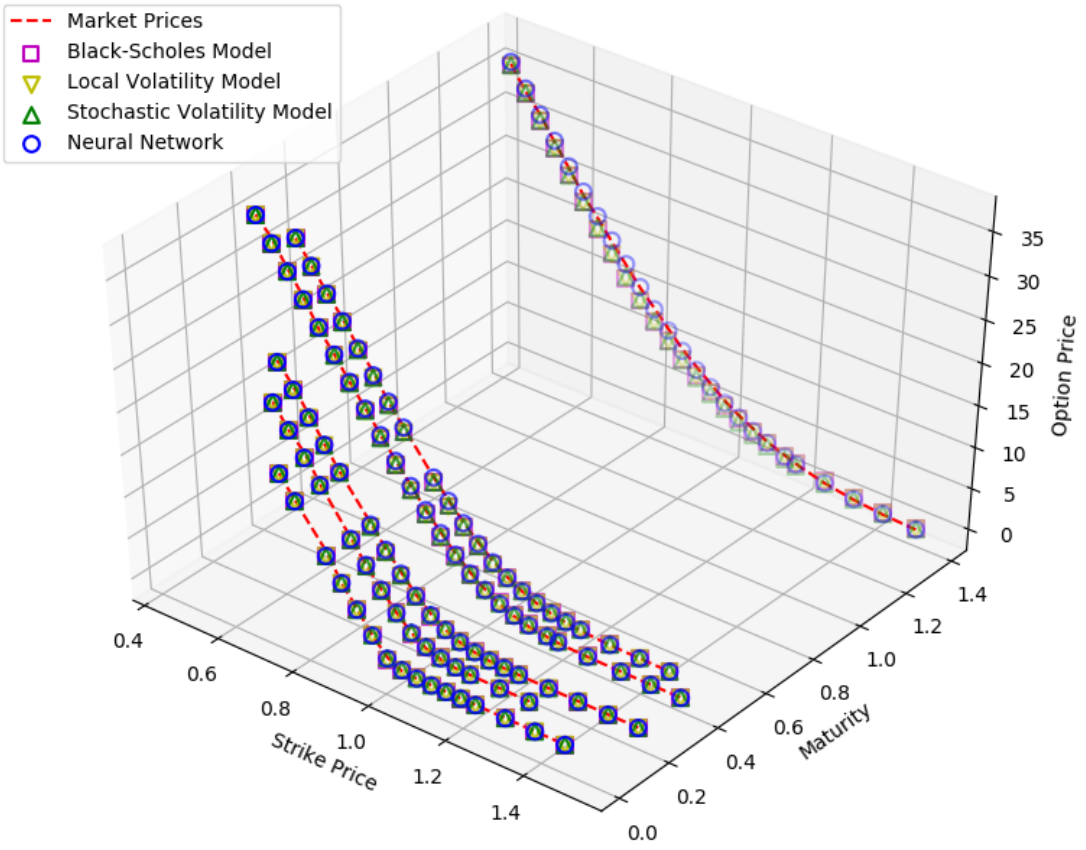


Figure 4.7: The next day out of sample call option curve for ticker "PCAR". The date is 2017-09-05. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

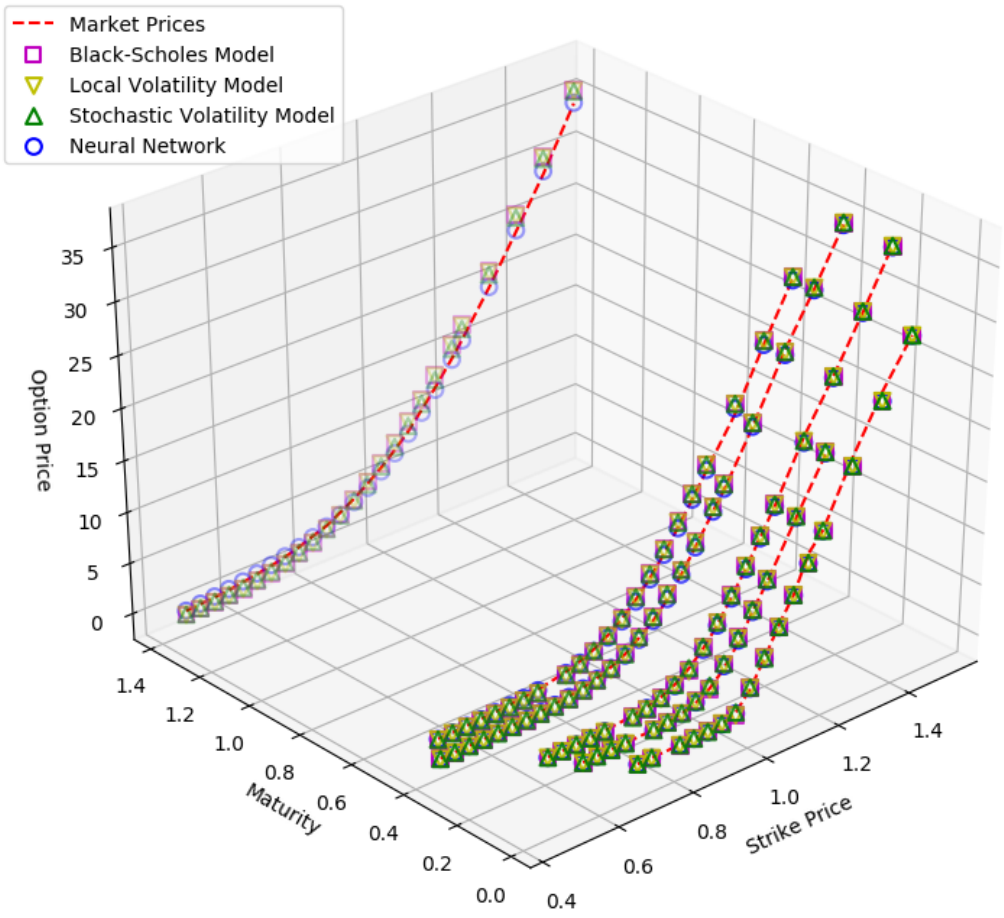


Figure 4.8: The next day out of sample put option curve for ticker "PCAR". The date is 2017-09-05. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

We can also get a similar conclusion as the intraday experiment which is the 2D-NN model calibrates the market price more accurately than the other three models, especially when the maturity is large.

Out-of-sample Payoff Functions

In this part, we still use the data in 2017-09-01. We train only on the available call options and predict the put options.

Experiment of price prediction, part 1.						
	BS Scalar		BS-LV		Heston	
	Call	Put	Call	Put	Call	Put
2017-09-01	0.450	0.531	0.258	0.415	0.412	0.492
	1.273	2.018	0.355	1.337	1.142	1.745
	53.433%	45.518%	36.061%	45.653%	35.262%	36.340%

Table 4.6: Experiment of the options price prediction. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of price prediction, part 2.				
	1D-NN		NNLV	
	Call	Put	Call	Put
2017-09-01	0.432	0.479	0.430	0.487
	1.262	1.517	1.196	1.528
	43.413%	38.215%	44.740%	38.576%

Table 4.7: Experiment of options price prediction. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of price prediction, part 3.				
	2D-NN-Heston		2D-NN	
	Call	Put	Call	Put
2017-09-01	0.361	0.420	0.317	0.488
	1.097	1.391	0.956	1.443
	31.903%	36.347%	27.744%	38.385%

Table 4.8: Experiment of options price prediction. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Below are selected figures to visualize the performance of these models.

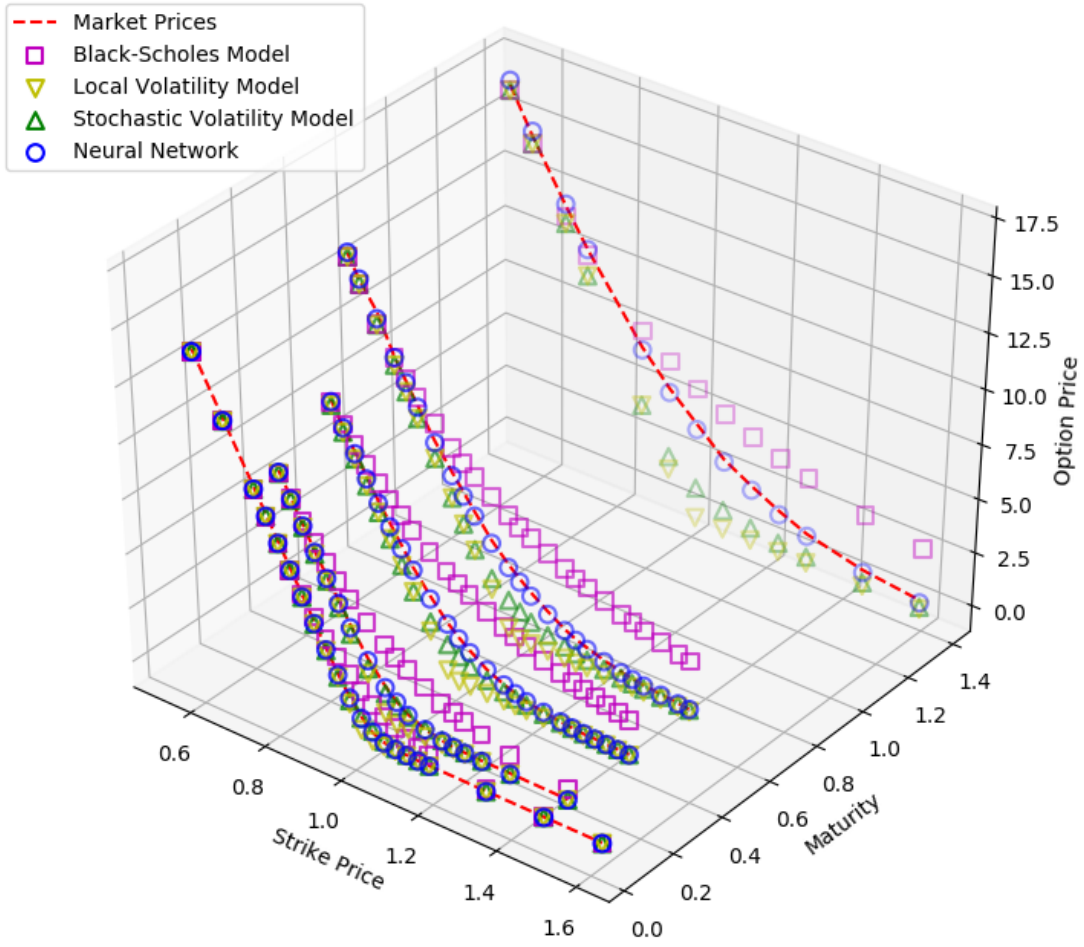


Figure 4.9: The out-of-sample payoff function experiment fitting the call option curve for ticker "BKR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

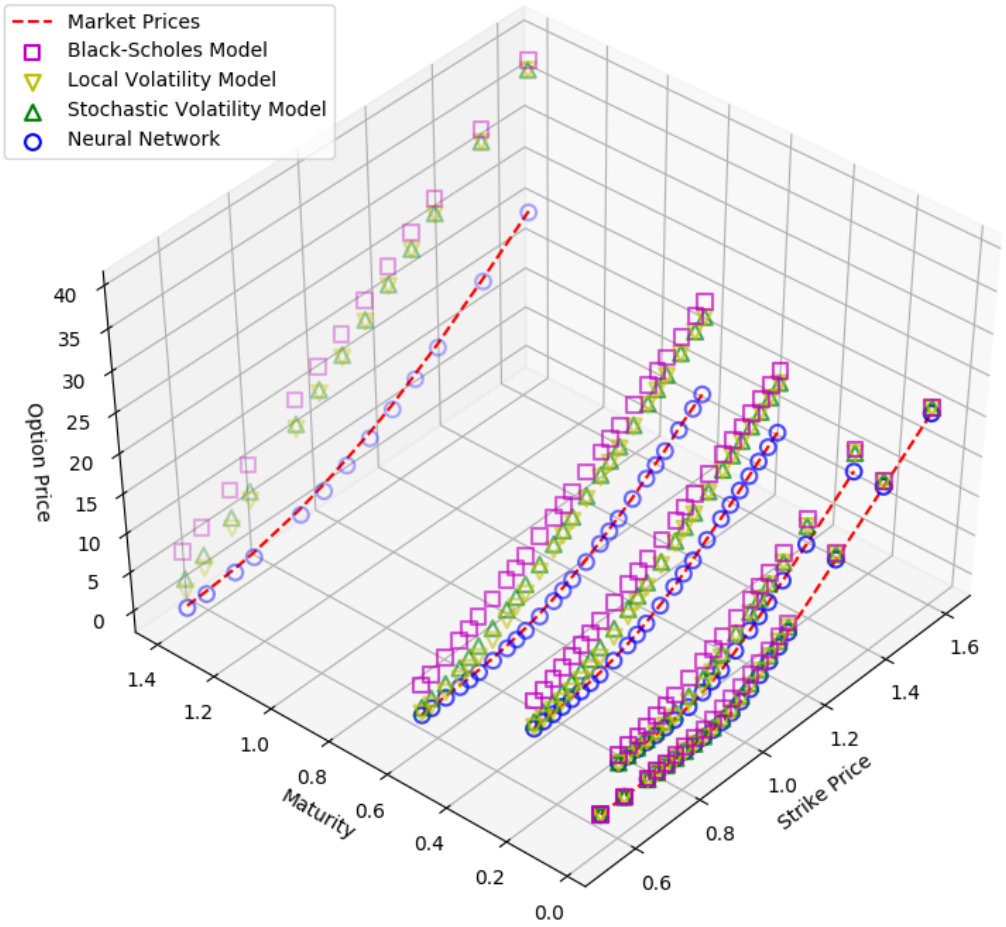


Figure 4.10: The out-of-sample payoff function experiment fitting the put option curve for ticker "BKR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

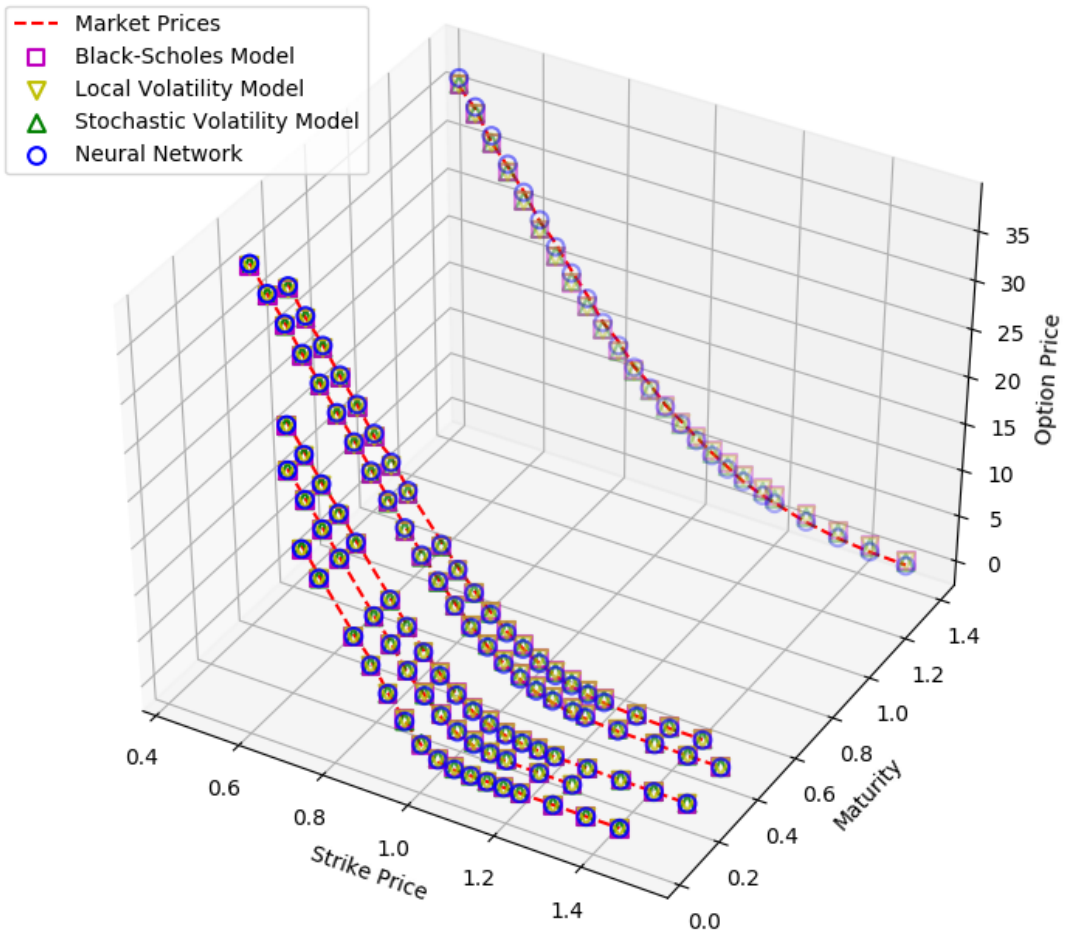


Figure 4.11: The out-of-sample payoff function experiment fitting the call option curve for ticker "PCAR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

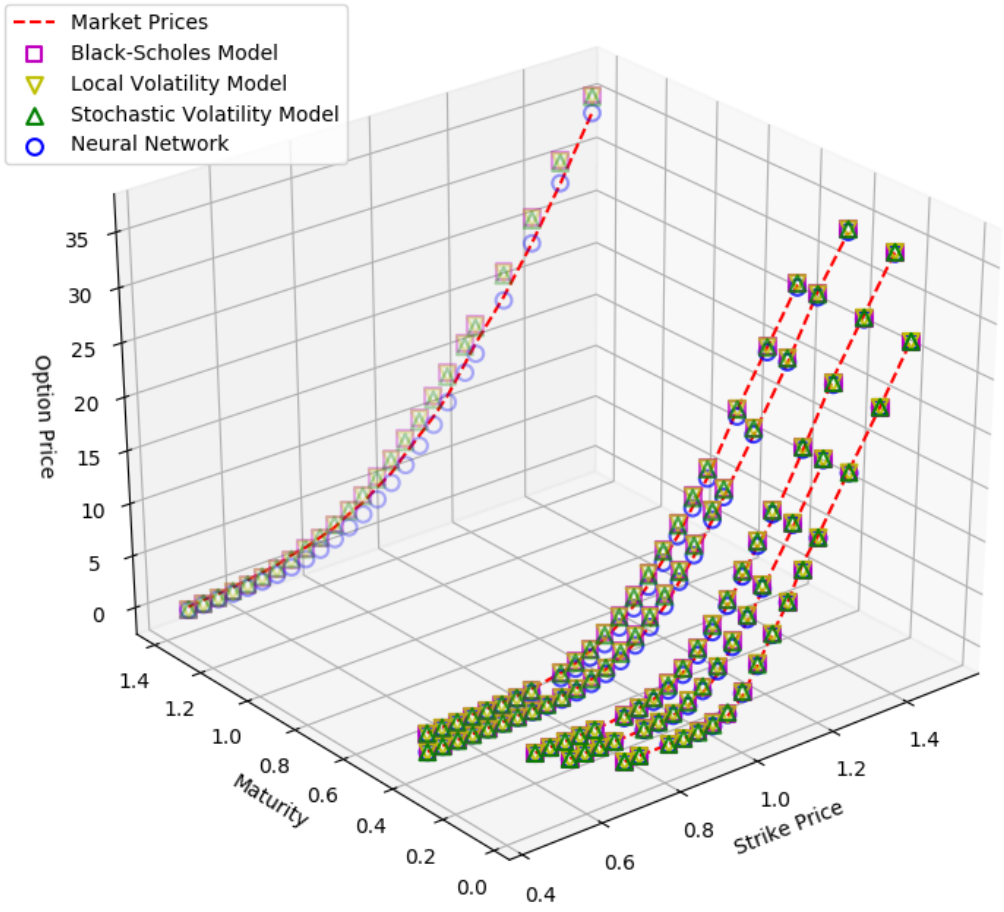


Figure 4.12: The out-of-sample payoff function experiment fitting the put option curve for ticker "PCAR". The date is 2017-09-01. "Market Prices" represents the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

We can also get a similar conclusion as the previous experiment which is the 2D-NN model calibrates the market price more accurately than the other three models, especially when the maturity is large.

4.5.2 S&P 100 Index Option

In this experiment, we select the 1-day S&P100 index data (2017-09-01). The S&P 100 index has two types of options.

- **OEX**: S&P 100 Index Options with American-style exercise.
- **XEO**: S&P 100 Index Options with European-style exercise.

The OEX options are traded more actively.

In our experiments below, we train the neural networks on the European options data and predict our model performance on the American options data.

Experiment for S&P 100 option, part 1.						
	BS Scalar		BS-LV		Heston	
	Call	Put	Call	Put	Call	Put
2017-09-01	2.062	1.773	1.893	1.555	2.132	1.789
	13.141	10.835	10.020	7.706	10.542	9.200
	42.600%	65.815%	49.956%	75.016%	63.313%	67.480%

Table 4.9: Experiment of training on European options and test on American options for PDE model. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment for S&P 100 option, part 2.				
	1D-NN		NNLV	
	Call	Put	Call	Put
2017-09-01	2.035	1.812	1.963	1.653
	11.540	10.019	10.095	8.723
	51.768%	69.681%	42.910%	63.484%

Table 4.10: Experiment of training on European options and test on American options-for PDE model. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment for S&P 100 option, part 3.				
	2D-NN-Heston		2D-NN	
	Call	Put	Call	Put
2017-09-01	1.904 8.339 45.455%	1.649 9.951 70.487%	2.030 8.064 35.079%	1.336 4.665 65.195%

Table 4.11: Experiment of training on European options and test on American options-for PDE model. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Here we select the maturity date that equals 476 days and plots the 2-dimensional figures below.

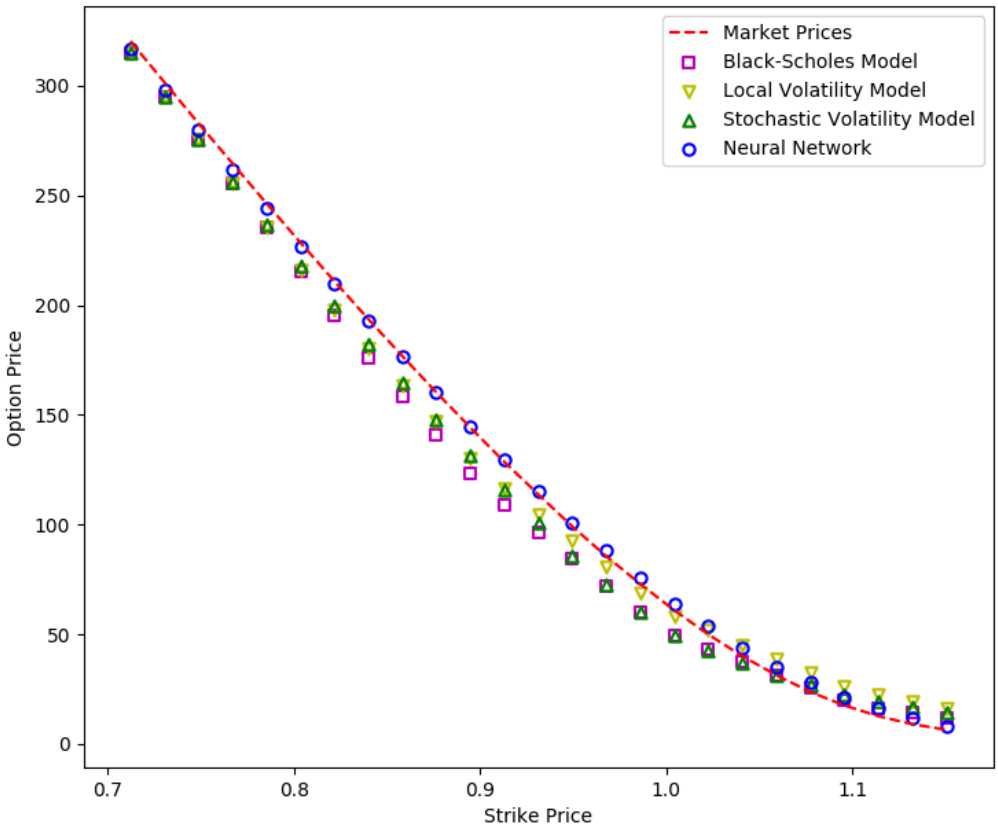


Figure 4.13: The out-of-sample payoff function experiment fitting the call option curve for the S&P100 index option when the maturity is 476 days. The date is 2017-09-01. "Market Prices" represents for the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

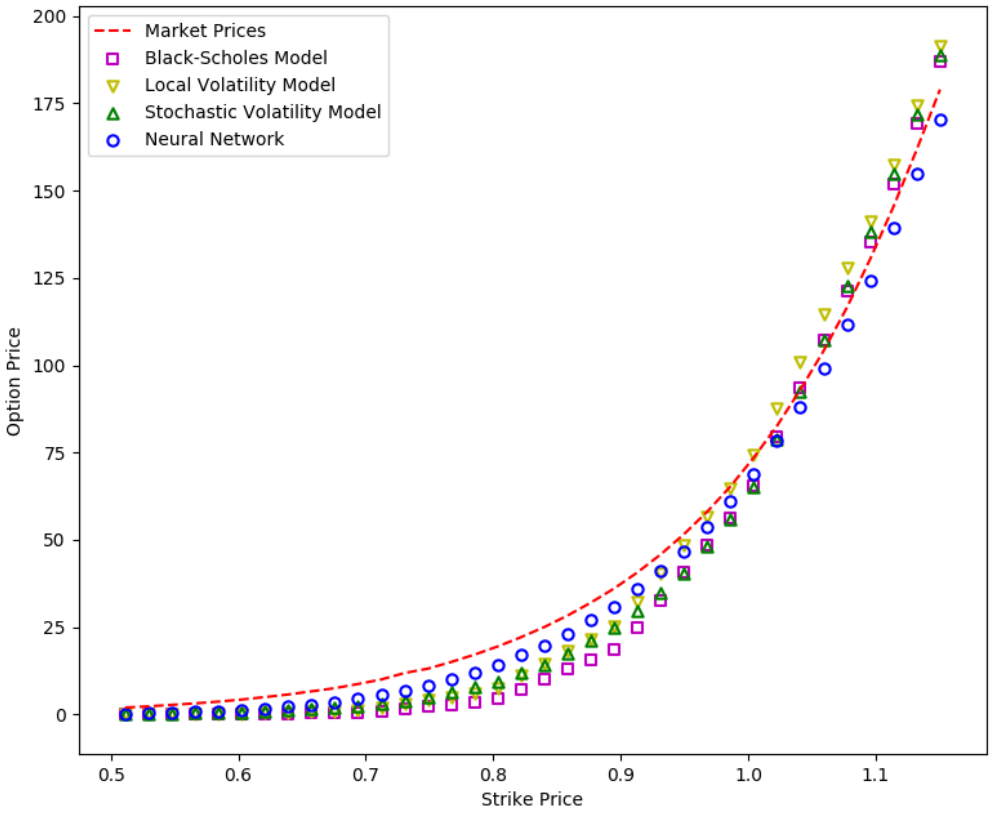


Figure 4.14: The out-of-sample payoff function experiment fitting the put option curve for the S&P100 index option when the maturity is 476 days. The date is 2017-09-01. "Market Prices" represents for the true option prices. "Black-Scholes Model" means the "BS-Scalar" model. "Local Volatility Model" means the "NNLV" model. "Neural Network" represents the "2D-NN", while the "Stochastic Volatility Model" represents the Heston model.

We can easily observe that the 2D-NN model calibrates the market price more accurately than the other three models when the maturity is large.

Chapter 5

Delta Hedging for American Options

Now let's discuss the delta hedging for the American options. We have similar evaluation metrics as the delta hedging for European options which have been discussed in 3.1. The following sections are the experiments for hedging American options.

5.1 The Next-day Hedging

Below are the next-day hedging results corresponding to the experiment in 4.5.1. We take the average of the 100 stocks as the evaluation metrics.

Experiment of Hedging for the Next Day, part 1.						
2017-09-05	BS Scalar		BS-LV		Heston	
	Call	Put	Call	Put	Call	Put
MAE	0.169	0.167	0.164	0.159	0.174	0.170
MSE	0.091	0.111	0.087	0.102	0.101	0.119
relative MAE	24.541%	16.855%	23.326%	16.807%	25.101%	17.230%

Table 5.1: Experiment of hedging for the next day for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of Hedging for the Next Day, part 2.				
2017-09-05	1-to-1		LV-NN	
	Call	Put	Call	Put
MAE	0.187	0.181	0.187	0.181
MSE	0.126	0.143	0.126	0.143
relative MAE	26.757%	18.488%	26.715%	18.515%

Table 5.2: Experiment of hedging for the next day for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of Hedging for the Next Day, part 3.				
2017-09-05	2D-NN-Heston		2D-NN	
	Call	Put	Call	Put
MAE	0.174	0.172	0.176	0.173
MSE	0.104	0.124	0.107	0.125
relative MAE	22.909%	17.045%	23.881%	17.229%

Table 5.3: Experiment of hedging for the next day for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

We can observe that the neural-network-based PDE model outperforms the local volatility model while slightly under-performing the Black-Scholes model and the Heston model.

5.2 The Next 5th Day Hedging

Below are the results of hedging for the 5th day.

Experiment of Hedging for the Next 5th Day, part 1.						
2017-09-05	BS Scalar		BS-LV		Heston	
	Call	Put	Call	Put	Call	Put
MAE	0.387	0.363	0.382	0.361	0.348	0.326
MSE	1.112	1.263	1.058	1.206	0.952	1.067
relative MAE	36.870%	40.553%	34.670%	40.916%	36.581%	40.769%

Table 5.4: Experiment of hedging for the next 5th day for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of Hedging for the Next 5th Day, part 2.				
2017-09-05	1-to-1		LV-NN	
	Call	Put	Call	Put
MAE	0.364	0.342	0.364	0.341
MSE	1.089	1.209	1.093	1.211
relative MAE	39.382%	42.684%	39.382%	42.704%

Table 5.5: Experiment of hedging for the next 5th day for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

Experiment of Hedging for the Next 5th Day, part 3.				
2017-09-05	2D-NN-Heston		2D-NN	
	Call	Put	Call	Put
MAE	0.343	0.322	0.346	0.323
MSE	0.967	1.083	0.981	1.087
relative MAE	32.045%	40.267%	34.400%	40.601%

Table 5.6: Experiment of hedging for the next 5th day for stock options. There are three rows for each column. The first row is for "MAE". "MAE" means mean absolute error. The second row is for "MSE". "MSE" means mean squared error. The third row is for "relative MAE", which means the relative mean absolute error.

When the hedging period goes longer, the hedging performance decreases for all the models, especially for the Black-Scholes model. We can observe that the neural-network-based PDE model outperforms the Black-Scholes model and the local volatility model while slightly under-performing the Heston model.

Chapter 6

Deep Learning Models for High-frequency Data

In this chapter, we explore the application of machine learning models to high-frequency data. In particular, the objective is to predict the direction of the mid-price change at small time horizons (such as 1 second or 10^{-1} seconds) given a sequence of limit order book data. In order to avoid imbalanced labels (the mid-price frequently does not change at small time horizons), we predict the conditional probability that the price moves up or down conditional on the price changing. We use the cross-entropy error as the loss function.

6.1 Limit Order Books Data and Selected Features

A limit order book is a typical type of data in high-frequency trading. It mainly consists of two files:

- **The order book file:** In the column direction, it contains ten levels of information. Each level includes bid price, bid size, ask price and ask size. The

bid price refers to the price a buyer will pay, and the asking price refers to the price a seller will accept. In particular, level one is the place that transactions occur, where we have the best bid price (highest bid price) and the best ask price (lowest ask price). In the row direction, we have the tick data, which means we create a new row when any information in the ten levels is updated.

- **The message file:** It includes the time, event type, order ID, size, price, and direction of the order for each corresponding row in the order book file. Together with the order book file, we can observe any action that can be seen publicly (there are hidden orders) and their corresponding influence on order book files.

The dataset[1] we use consists of 3-year data of over 1200 common stocks in the market. The total size is over two terabytes.

The features which are inputs to the models are the first ten levels of the bid and ask sizes, stock price, spread, and previous mid-price direction.

6.2 Models

We developed several deep learning models and compared their performance against machine learning models such as random forest and support vector machines (SVM).

- **Deep convolutional neural networks (CNN):** The basic structure of the neural network architecture is from the paper [7]. After tuning the hyperparameters, the model contains 17 convolutional layers. The filter size is 3, and the number of filters for each layer ranges from 64 to 512. Techniques such as pooling, dropout, and residual networks are also included in the model.

[1]Source: https://lobsterdata.com/index.php

- **Long short-term memory (LSTM) model:** The model has 2 LSTM layers, with each layer containing 64 units. We use backpropagation through time (BPTT) to train the model.

- **Hybrid models:** The hybrid models are two CNN (LSTM) models that are updated using different timescale data (a data sequence of 1 second and a second data sequence of 0.1 seconds). The multiple timescales help address the vanishing gradient problem for learning sequential relationships over long periods.

Selected results on the 1-second dataset are presented in table 6.1 below. We can observe that neural networks perform better than traditional machine learning models (such as random forest and SVM), and the LSTM network has higher accuracy than the CNN. In addition, the hybrid model performs better than the LSTM network.

Accuracy of Machine Learning Models for Limit Order Books					
Stock Ticker	SVM	Random Forest	CNN	LSTM	2ScaleLSTM
AMD	0.600	0.727	0.791	0.809	0.823
BIDU	0.591	0.590	0.592	0.584	0.605
CBS	0.610	0.593	0.628	0.632	0.644
EMR	0.637	0.625	0.701	0.682	0.695
HON	0.598	0.569	0.639	0.626	0.633
JNPR	0.744	0.680	0.751	0.769	0.783
RCL	0.570	0.555	0.612	0.605	0.601
UPS	0.592	0.559	0.622	0.615	0.623

Table 6.1: Comparing the accuracy of different models for limit order books on 1-second data. '2ScaleLSTM' is a hybrid model that additionally uses 0.1-second data.

We also compared two LSTM models. In the first model, we make predictions with the cell states at each timestamp while in the second model we wait until the end of the sequence and use the last cell state to make predictions. We call the former one-to-one model and the latter many-to-one model. A comparison of the two approaches is presented in the table below.

Accuracy of LSTM Models for Limit Order Books		
Stock Ticker	One-to-one	Many-to-one
AAPL	0.6926	0.6889
ACN	0.6262	0.6191
BA	0.6164	0.6141
CMCSA	0.7626	0.7542
DIS	0.6848	0.6824
JPM	0.6947	0.6919
MMM	0.6304	0.6268
MSFT	0.7978	0.7974
PM	0.6632	0.6581
V	0.7099	0.7054

Table 6.2: Comparing the accuracy of one-to-one and many-to-one LSTM models for limit order books on 1-second data.

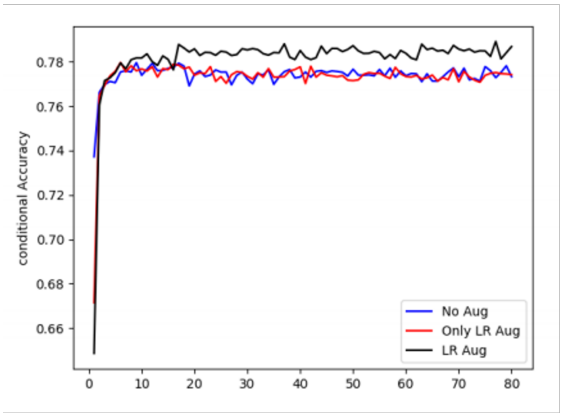
6.3 Data Augmentation

The performance of machine learning models is generally limited by the number of data points. However, we can generate new “artificial" data samples using data augmentation, which effectively increases the size of the training set and therefore reduces overfitting. Well-known data augmentation methods in image recognition are rotating or flipping images. We have developed data augmentation methods for high-frequency financial data.

- **Flip augmentation:** The idea is to swap the ask and bid sizes, and reverse the direction of the price changes.
- **Time reversal augmentation:** We reverse the data along the time direction so that the last row becomes the first one. The price change labels are updated correspondingly.

In our experiments, flip augmentation further improved the accuracy of deep learning models trained on small to medium size datasets.

Flip augmentation:



Time reversal augmentation:

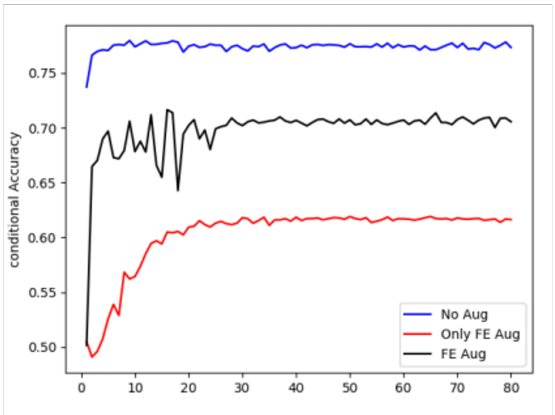


Figure 6.1: Results for data augmentation. The left side is the flip augmentation. The right side is the time reversal augmentation. The blue line in the figures refers to the result without augmentation. The red line refers to the result that only has augmentation. And the black line means we use both original and augmented data.

6.4 Ensemble Models

We have also evaluated ensemble methods for improving the accuracy of deep learning models of financial data. Ensemble models train a number of deep learning models and then average their predictions, which typically leads to reducing overfitting. In our experiments, the ensemble model performed better than a single neural network and could achieve higher out-of-sample prediction accuracy.

Accuracy of Machine Learning Models for Limit Order Books					
Stock Ticker	AAPL	AMZN	COST	UNH	WMT
LSTM 1	0.685414	0.611214	0.634820	0.616830	0.707776
LSTM 2	0.686829	0.611960	0.635189	0.616373	0.705319
LSTM 3	0.684512	0.610716	0.636008	0.614917	0.711526
LSTM 4	0.686736	0.609940	0.637333	0.615857	0.708613
LSTM 5	0.685565	0.611492	0.634639	0.614647	0.707956
LSTM 6	0.686524	0.607422	0.637061	0.616269	0.709399
LSTM 7	0.686635	0.609394	0.634906	0.614874	0.709864
LSTM 8	0.687294	0.610550	0.633179	0.608249	0.707171
Ensemble	0.689094	0.613461	0.638481	0.618831	0.711088

Table 6.3: Comparing the ensemble performance for the LSTM model.

The table lists the conditional accuracy of 8 independent LSTM models and the ensembled model on 5 stocks for the many-to-one model. We can see from the table that all 8 models have a close prediction accuracy and the ensembled model outperforms all of the individual ones.

Chapter 7

Conclusion

In this thesis, we develop a neural network SDE and PDE approach to pricing and hedging European and American options. Compared to traditional mathematical options pricing models, the neural-network-based local volatility models and the 2-dimensional neural-network models achieve superior performance. We also discuss their delta hedging performance.

Again, we would like to list the main conclusions for this thesis.

- Propose neural-network-based SDE models that demonstrate improved pricing accuracy compared to traditional mathematical option pricing models.
- Develop a new stochastic gradient descent (SGD) algorithm for calibrating neural-network SDE models given European option pricing data.
- Develop a calibration approach using the PDE representation for the neural-network SDE model.
- The neural-network-based PDE methods allow us to calibrate the neural-network SDE model to American option market pricing data.

- For out-of-the-money hedging, the neural-network-based SDE model outperforms Dupire's local volatility model and the Heston model while slightly under-performing the Black-Scholes model.
- For at-the-money hedging, the neural-network-based SDE model underperforms the other three models.
- In terms of hedging for the PDE models, the neural-network-based model outperforms the local volatility model while slightly under-performing the Black-Scholes model and the Heston model.
- The recalibration method is effective on both pricing and hedging.
- Develop and evaluate deep learning models for high-frequency limit order book data.

In summary, we develop a neural-network-based framework to price European and American options. A new optimization algorithm is proposed that can take unbiased, stochastic gradient descent steps for optimizing the neural-network-based SDE for mean-squared-error objective functions and European options. This algorithm not only outperforms the traditional options pricing models but also allows for rapid optimization of the models. In addition, for training with general objective functions and more complex options (e.g., American options), a partial differential equation (PDE) is derived to evaluate the objective function and we then optimize over the PDE. Moreover, their hedging performance and recalibration performance over long time period is also discussed. All numerical results are based on real market data including SP500 index options, SP100 index options, and single-stock American options.

Bibliography

[1] Avellaneda, M., and Stoikov, S. High-frequency trading in a limit order book. *Quantitative Finance 8*, 3 (2008), 217–224.

[2] Biais, B., Hillion, P., and Spatt, C. An empirical analysis of the limit order book and the order flow in the paris bourse. *the Journal of Finance 50*, 5 (1995), 1655–1689.

[3] Black, F., and Scholes, M. The pricing of options and corporate liabilities. *Journal of political economy 81*, 3 (1973), 637–654.

[4] Boyle, P. P. Options: A monte carlo approach. *Journal of financial economics 4*, 3 (1977), 323–338.

[5] Brennan, M. J., and Schwartz, E. S. Finite difference methods and jump processes arising in the pricing of contingent claims: A synthesis. *Journal of Financial and Quantitative Analysis* (1978), 461–474.

[6] Carverhill, A. P., and Cheuk, T. H. Alternative neural network approach for option pricing and hedging. *Available at SSRN 480562* (2003).

[7] Conneau, A., Schwenk, H., Barrault, L., and Lecun, Y. Very deep convolutional networks for text classification. *arXiv preprint arXiv:1606.01781* (2016).

[8] Cox, J. C., Ross, S. A., and Rubinstein, M. Option pricing: A simplified approach. *Journal of financial Economics 7*, 3 (1979), 229–263.

[9] Cybenko, G. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems 2*, 4 (1989), 303–314.

[10] Davis, M. H. The dupire formula.

[11] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition* (2009), Ieee, pp. 248–255.

[12] Derman, E., and Kani, I. Stochastic implied trees: Arbitrage pricing with stochastic term and strike structure of volatility. *International journal of theoretical and applied finance 1*, 01 (1998), 61–110.

[13] Dupire, B., et al. Pricing with a smile. *Risk 7*, 1 (1994), 18–20.

[14] Funahashi, H., and Uzawa, K. Artificial neural network for option pricing with and without asymptotic correction. *Available at SSRN 3461104* (2019).

[15] Gierjatowicz, P., Sabate-Vidales, M., Siska, D., Szpruch, L., and Zuric, Z. Robust pricing and hedging via neural sdes. *Available at SSRN 3646241* (2020).

[16] Goswami, A., Rajani, S., and Tanksale, A. Data-driven option pricing using single and multi-asset supervised learning. *arXiv preprint arXiv:2008.00462* (2020).

[17] Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., and Howison, S. D. Limit order books. *Quantitative Finance 13*, 11 (2013), 1709–1742.

[18] Healy, J., Dixon, M., Read, B., and Cai, F. A data-centric approach to understanding the pricing of financial options. *The European Physical Journal B-Condensed Matter and Complex Systems 27*, 2 (2002), 219–227.

[19] Herzel, S. Option pricing with stochastic volatility models. *Decisions in Economics and Finance 23*, 2 (2000), 75–99.

[20] Heston, S. L. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The review of financial studies 6*, 2 (1993), 327–343.

[21] Hornik, K., Stinchcombe, M., and White, H. Multilayer feedforward networks are universal approximators. *Neural networks 2*, 5 (1989), 359–366.

[22] Hutchinson, J. M., Lo, A. W., and Poggio, T. A nonparametric approach to pricing and hedging derivative securities via learning networks. *The Journal of Finance 49*, 3 (1994), 851–889.

[23] Kelly, D. L., Shorish, J., et al. Valuing and hedging american put options using neural networks. *Unpublished manuscript, Carnegie Mellon University* (1994).

[24] Kolmogoroff, A. Über die analytischen methoden in der wahrscheinlichkeitsrechnung. *Mathematische Annalen 104*, 1 (1931), 415–458.

[25] Kou, S. G. A jump-diffusion model for option pricing. *Management science 48*, 8 (2002), 1086–1101.

[26] Krizhevsky, A., Nair, V., and Hinton, G. Cifar-10 (canadian institute for advanced research). *URL http://www. cs. toronto. edu/kriz/cifar. html 5* (2010).

[27] LeCun, Y., Bengio, Y., and Hinton, G. Deep learning. *nature 521*, 7553 (2015), 436–444.

[28] Long, W., Lu, Z., and Cui, L. Deep learning-based feature engineering for stock price movement prediction. *Knowledge-Based Systems 164* (2019), 163–173.

[29] MacKenzie, D. *An engine, not a camera: How financial models shape markets*. Mit Press, 2008.

[30] Malliaris, M., and Salchenberger, L. Beating the best: A neural network challenges the black-scholes formula. In *Proceedings of 9th IEEE Conference on Artificial Intelligence for Applications* (1993), IEEE, pp. 445–449.

[31] Malliaris, M., and Salchenberger, L. A neural network model for estimating option prices. *Applied Intelligence 3*, 3 (1993), 193–206.

[32] Merton, R. C. Theory of rational option pricing. *The Bell Journal of economics and management science* (1973), 141–183.

[33] Merton, R. C. Option pricing when underlying stock returns are discontinuous. *Journal of financial economics 3*, 1-2 (1976), 125–144.

[34] Rouah, F. D. *The Heston model and its extensions in Matlab and C*. John Wiley & Sons, 2013.

[35] Samuel, A. L. Some studies in machine learning using the game of checkers. *IBM Journal of research and development 3*, 3 (1959), 210–229.

[36] Schwartz, E. S. The valuation of warrants: Implementing a new approach. *Journal of Financial Economics 4*, 1 (1977), 79–93.

[37] Sirignano, J. A. Deep learning for limit order books. *Quantitative Finance 19*, 4 (2019), 549–570.

[38] Stein, E. M., and Stein, J. C. Stock price distributions with stochastic volatility: an analytic approach. *The review of financial studies 4*, 4 (1991), 727–752.